Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

ОТЧЁТ

о лабораторной работе № 1

«Искусственная жизнь» — программа, использующая логических агентов

по дисциплине «Объектно-ориентированное программирование на языке Java»

Выполнил
студент гр. _____ _____ А. А. Минязов

Проверил
старший преподаватель _____ Д. С. Хасанов

Санкт-Петербург
2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
1.1 Цель работы 3
1.2 Декомпозиция задачи 4
2 Описание модели 5
2.1 Среда 5
2.2 Агенты 5
2.3 Энергия, гибель и размножение 5
2.4 Логика поведения 6
2.5 Шаг времени 6
2.6 Параметры и их подбор 6
3 Проектирование 9
3.1 Диаграмма прецедентов 9
3.2 Диаграммы деятельности 10
3.3 Диаграммы классов 14
3.4 Диаграмма последовательности 17
3.5 Диаграмма компонентов 18
4 Реализация 19
4.1 Средства разработки 19
4.2 Структура программы 19
4.3 Применение принципов ООП 19
4.4 Консольная версия 20
4.5 Оконная версия 20
4.6 Тестирование 21
5 Результаты работы 23
5.1 Консольная версия 23
5.2 Оконная версия 25
Заключение 28
Приложение А. Текст программы 29

1 Постановка задачи

Требуется разработать программу, реализующую алгоритм искусственной жизни. Алгоритм моделирует пищевую цепочку для логических агентов трёх видов: «хищники», «травоядные» и «растения». Агенты помещены в искусственную среду, представленную двумерным массивом ячеек. В ячейке может находиться один из агентов, либо ячейка может быть пустой.

Каждый агент имеет некоторое количество единиц энергии. Энергия хищника и травоядного уменьшается с каждым шагом игры. Чтобы увеличить энергию, хищник поедает травоядного, а травоядное поедает растение; при этом энергия агента увеличивается на заданное число. Если энергия агента равна нулю, он погибает. Если энергия превышает порог, появляется новый агент, который забирает часть энергии.

Травоядные и хищники обладают зрением и могут перемещаться. Область видимости составляет 5×5 клеток, агент находится в центре. Действия агентов основаны на логических выражениях.

Необходимо создать классы окружающей среды, растения, травоядного, хищника и другие необходимые классы; класс агента должен включать данные и методы действий: движение, поедание, появление, исчезновение. Параметры программы нужно подобрать так, чтобы жизнь не прекращалась. Предварительный вариант программы — консольное приложение, окончательный — оконное приложение.

1.1 Цель работы

Цель работы — создать симуляцию экосистемы, в которой автономные логические агенты взаимодействуют друг с другом в рамках пищевой цепочки по строгим алгоритмическим правилам, и освоить на её примере основные принципы объектно-ориентированного программирования на языке Java: абстракцию, инкапсуляцию, наследование и полиморфизм.

1.2 Декомпозиция задачи

Задача разбита на четыре подсистемы:

- **пространственная модель (среда)** — двумерная сетка W×H, в клетке не более одного агента;
- **сущностная модель (агенты)** — три вида агентов с общим предком и собственным поведением;
- **физическая модель (энергия и метаболизм)** — расход, пополнение, гибель и размножение делением;
- **когнитивная модель (зрение и логика)** — выбор действия по содержимому области 5×5.

2 Описание модели

2.1 Среда

Среда — прямоугольное поле из клеток со стенами по краям (по умолчанию 100×60). Поле хранится в двумерном массиве Agent[][]: в клетке находится ссылка на агента или пустое значение null. Правило «в клетке не более одного агента» проверяется при каждом размещении и перемещении; координаты агента изменяет только поле, поэтому данные поля и агента всегда согласованы.

2.2 Агенты

- **Растение** не перемещается. Каждый шаг получает энергию «от солнца» (не выше заданного максимума) и при избытке энергии разрастается в соседнюю свободную клетку.
- **Травоядное** перемещается, видит область 5×5, питается растениями и убегает от хищников.
- **Хищник** перемещается, видит область 5×5 и охотится на травоядных.

Животные ходят на одну клетку за шаг в любом из восьми направлений, в том числе по диагонали.

2.3 Энергия, гибель и размножение

- **Метаболизм:** каждый шаг энергия животного уменьшается на расход своего вида; у растения расход равен нулю.
- **Питание:** съев добычу, животное получает фиксированную энергию своего вида и занимает клетку добычи.
- **Гибель:** если энергия животного стала равна нулю, оно погибает и удаляется с поля. Растение погибает, когда его съедают.
- **Размножение (деление):** если энергия агента строго больше порога его вида и рядом есть свободная клетка, в случайной свободной соседней клетке появляется потомок того же вида, получающий

половину энергии родителя. Суммарная энергия при делении сохраняется.

2.4 Логика поведения

Поведение животных задано логическими выражениями. Травоядное:

- ЕСЛИ видит хищника — делает шаг в свободную клетку, наиболее удалённую от ближайшего хищника;
- ИНАЧЕ ЕСЛИ видит растение — делает шаг к ближайшему растению и съедает его, если шаг приходится на клетку с растением;
- ИНАЧЕ — делает шаг в случайную свободную соседнюю клетку.

Хищник:

- ЕСЛИ видит травоядное — делает шаг к ближайшему травоядному и съедает его, если шаг приходится на его клетку;
- ИНАЧЕ — делает шаг в случайную свободную соседнюю клетку.

Перед выбором действия животное тратит энергию за ход, после действия — пытается размножиться. Если приблизиться к цели или отдалиться от угрозы нельзя, животное остаётся на месте. При нескольких одинаково подходящих вариантах выбор случайный.

2.5 Шаг времени

За один шаг каждый агент, живой на начало шага, делает один ход. Порядок ходов каждый раз перемешивается случайным образом, чтобы ни один агент не получал постоянного преимущества. Агент, съеденный до своей очереди, не ходит; потомок, родившийся на этом шаге, начинает ходить со следующего. Все случайные решения принимаются одним генератором случайных чисел, который создаётся из зерна (seed) в параметрах, поэтому при одинаковом seed ход симуляции полностью повторяется.

2.6 Параметры и их подбор

Параметры с наугад выбранными значениями приводили к колебаниям «бум и крах»: травоядные быстро размножались и выедали растения, хищники

размножались ещё быстрее и истребляли травоядных, после чего один из видов вымирал на 39–165-м шаге.

Параметры подобраны автоматическим перебором. Сначала проверено 600 случайных наборов (по 3 прогона на 1500 шагов); затем 12 лучших наборов в течение шести раундов улучшались небольшими случайными изменениями (по 6 прогонов на 3000 шагов); пять финалистов проверены на 20 новых начальных расстановках по 10 000 шагов. Лучший набор (таблица 1) сохранил все три вида во всех 20 прогонах.

Таблица 1 — Параметры по умолчанию

Параметр	Растение	Травоядное	Хищник
Количество в начале	2000	270	30
Начальная энергия	3	13	52
Изменение энергии за шаг	+3 (не выше 12)	−1	−2
Энергия от пищи	—	+14	+13
Порог размножения	9	26	103

Размер поля — 100×60 клеток, seed по умолчанию — 42. Найденные значения сдерживают «бум» хищников (для деления хищнику нужно съесть около восьми травоядных, а без пищи он быстро теряет энергию) и ускоряют восстановление растений, поэтому травоядные не успевают выесть их полностью.

Итоговые параметры дополнительно проверены на 40 начальных расстановках, не участвовавших в подборе: во всех 40 прогонах по 10 000 шагов все три вида сохранились; наименьшая численность травоядных — 179, хищников — 30. Проверка устойчивости (каждый параметр изменялся на ±20 %, по 8 прогонов на 5000 шагов) показала, что баланс чувствителен к трём параметрам (таблица 2); при остальных изменениях сохранялись все виды.

Таблица 2 — Параметры, к изменению которых чувствителен баланс

Изменение	Прогонов без вымирания
Порог размножения растения −20 % (9 → 7)	3 из 8

Изменение	Прогонов без вымирания
Энергия хищника от травоядного +20 % (13 → 16)	6 из 8
Прирост энергии растения +20 % (3 → 4)	7 из 8
Остальные параметры ±20 %	8 из 8

Вывод: опасна не только «трудная», но и слишком «лёгкая» жизнь — избыток пищи усиливает колебания численности и приводит к вымиранию.

3 Проектирование

Система описана пятью видами UML-диаграмм: прецедентов, деятельности, классов, последовательности и компонентов. Диаграммы построены в PlantUML; их исходные тексты хранятся в каталоге docs/uml проекта.

3.1 Диаграмма прецедентов

Диаграмма прецедентов (рисунок 1) показывает, что может делать пользователь: запускать и приостанавливать симуляцию, выполнять один шаг или (в консоли) N шагов, сбрасывать поле, менять скорость, настраивать параметры и наблюдать поле, счётчики и график. Настройка параметров всегда включает их проверку; при ошибках пользователю показываются сообщения и подсвечиваются неверные поля.

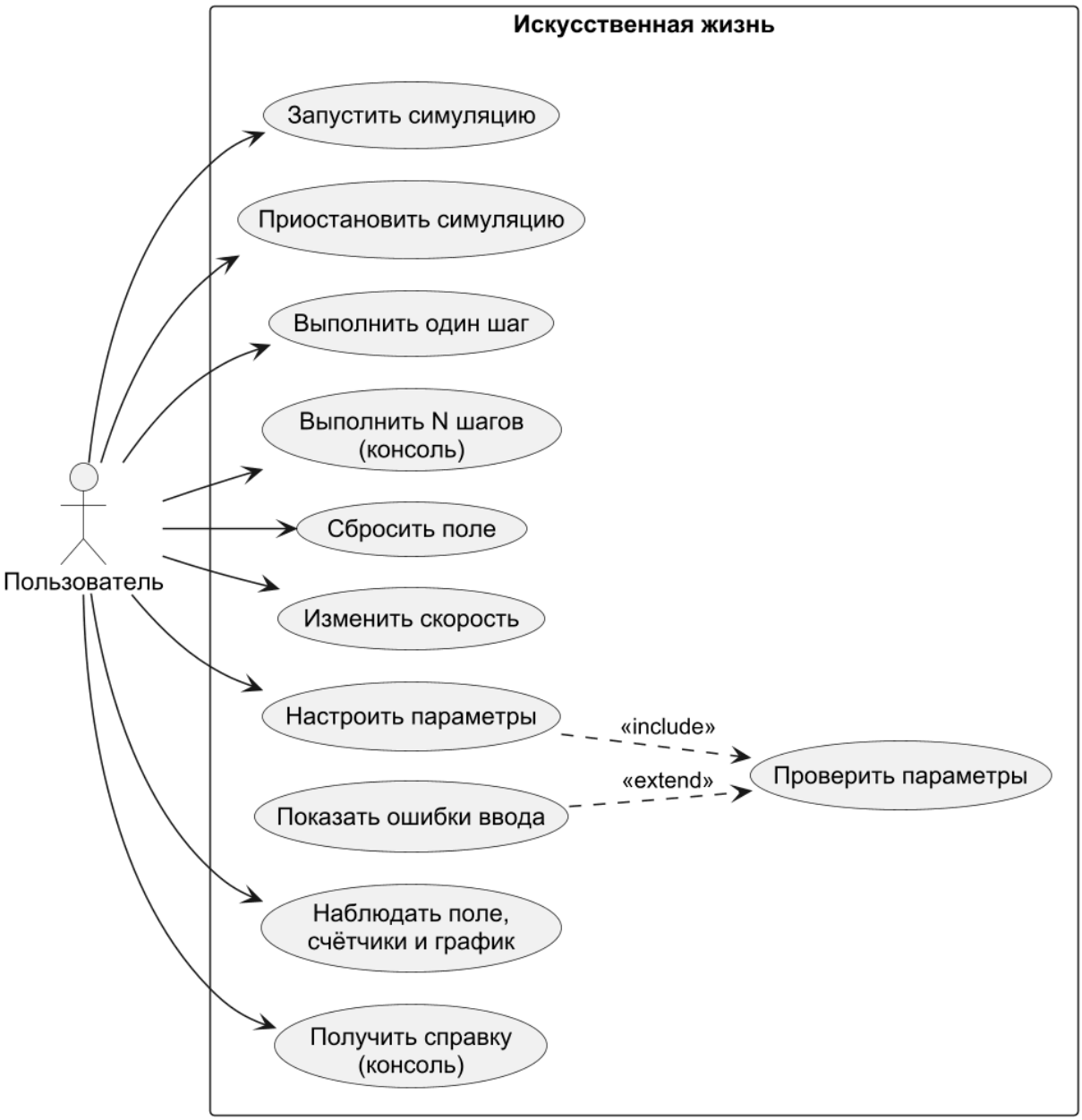


Рисунок 1 — Диаграмма прецедентов

3.2 Диаграммы деятельности

Алгоритм хода каждого вида агентов показан отдельной диаграммой деятельности (рисунки 2–4). Диаграммы соответствуют методам act() классов Plant, Herbivore и Predator.

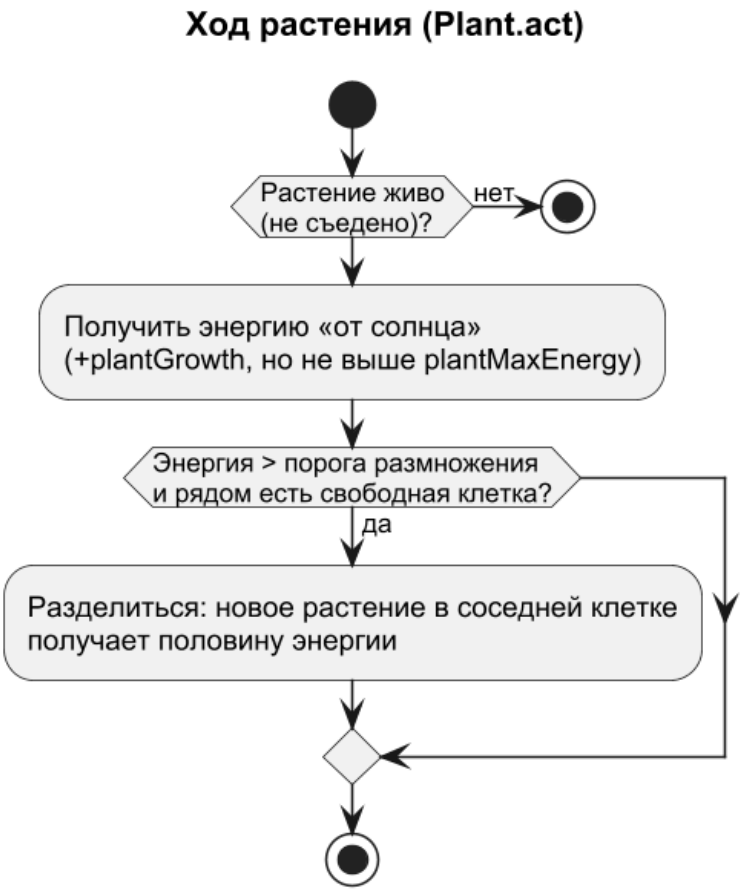


Рисунок 2 — Диаграмма деятельности: ход растения

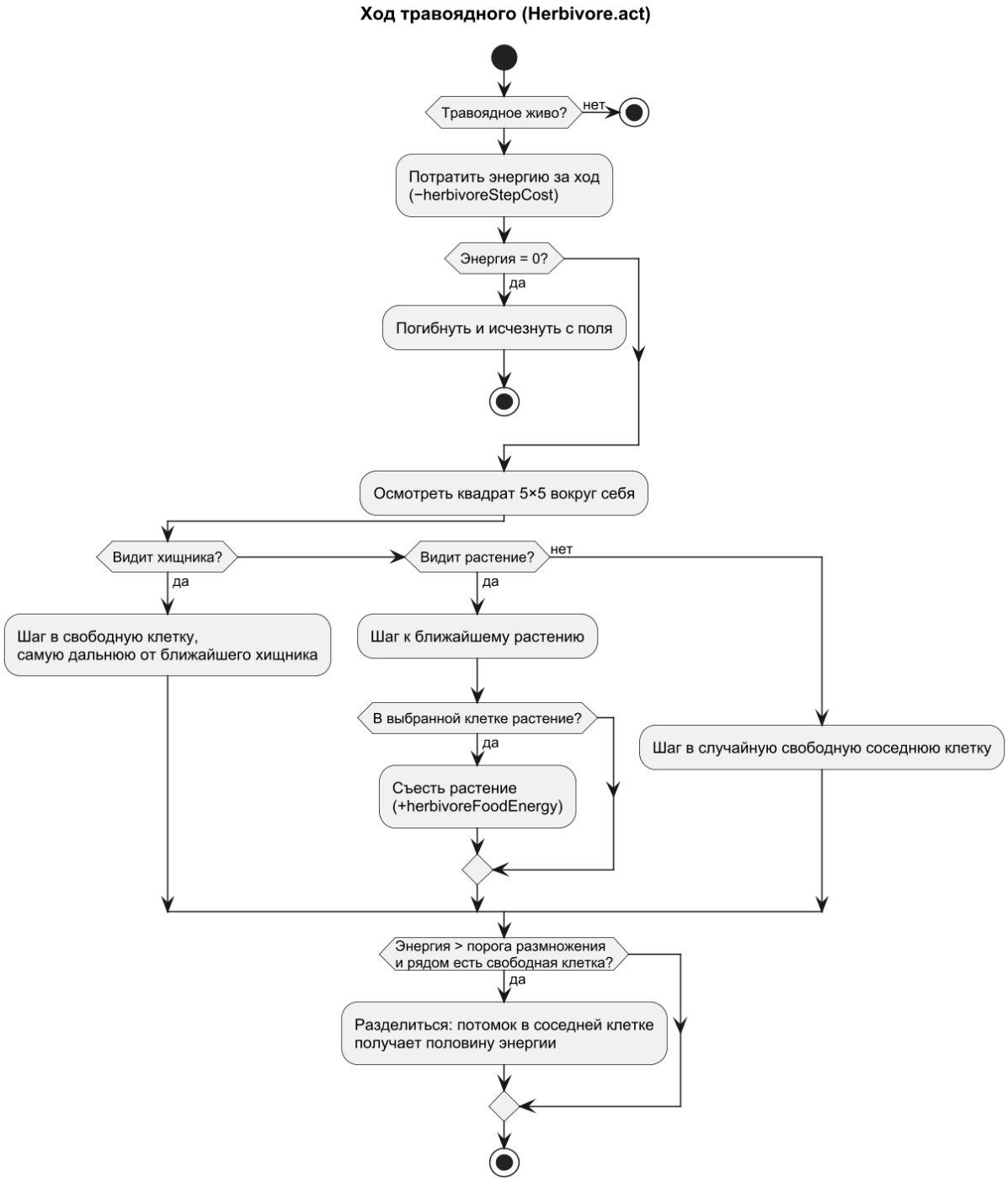


Рисунок 3 — Диаграмма деятельности: ход травоядного

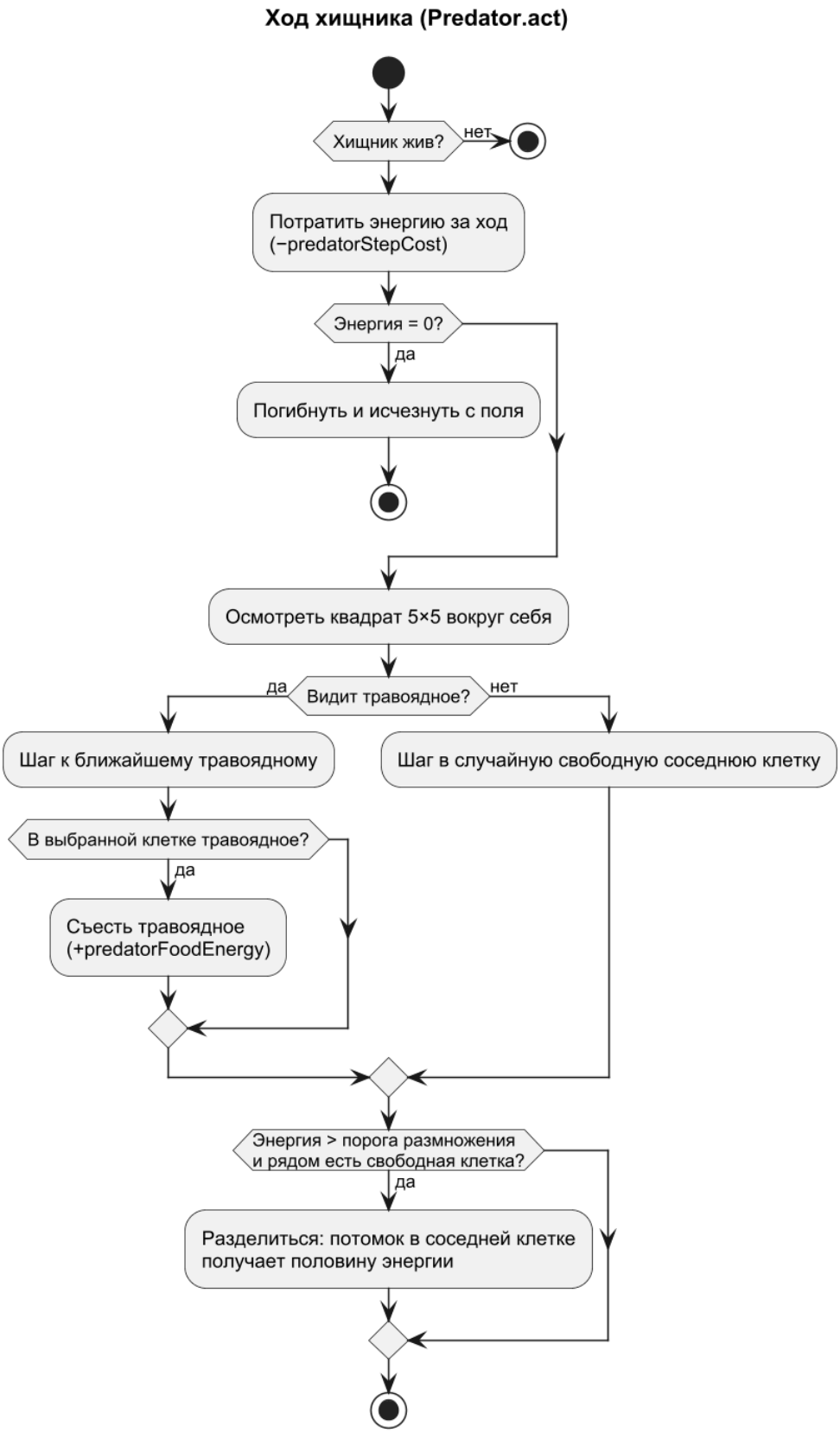


Рисунок 4 — Диаграмма деятельности: ход хищника

3.3 Диаграммы классов

Классы модели показаны на рисунке 5. Абстрактный класс Agent хранит общие для всех агентов данные (позицию, энергию, признак жизни) и общие действия (изменение энергии, гибель, размножение делением), а метод хода act() и способ создания потомка объявляет абстрактными. От него наследуются растение Plant и абстрактное животное Animal, которое добавляет зрение 5×5, перемещение, поедание и расход энергии. Травоядное Herbivore и хищник Predator реализуют только собственную логику и определение пищи. Класс World реализует интерфейс ReadOnlyWorld, через который интерфейс пользователя получает поле только для чтения. Параметры собраны в неизменяемой записи Settings, которая проверяет свои значения при создании.

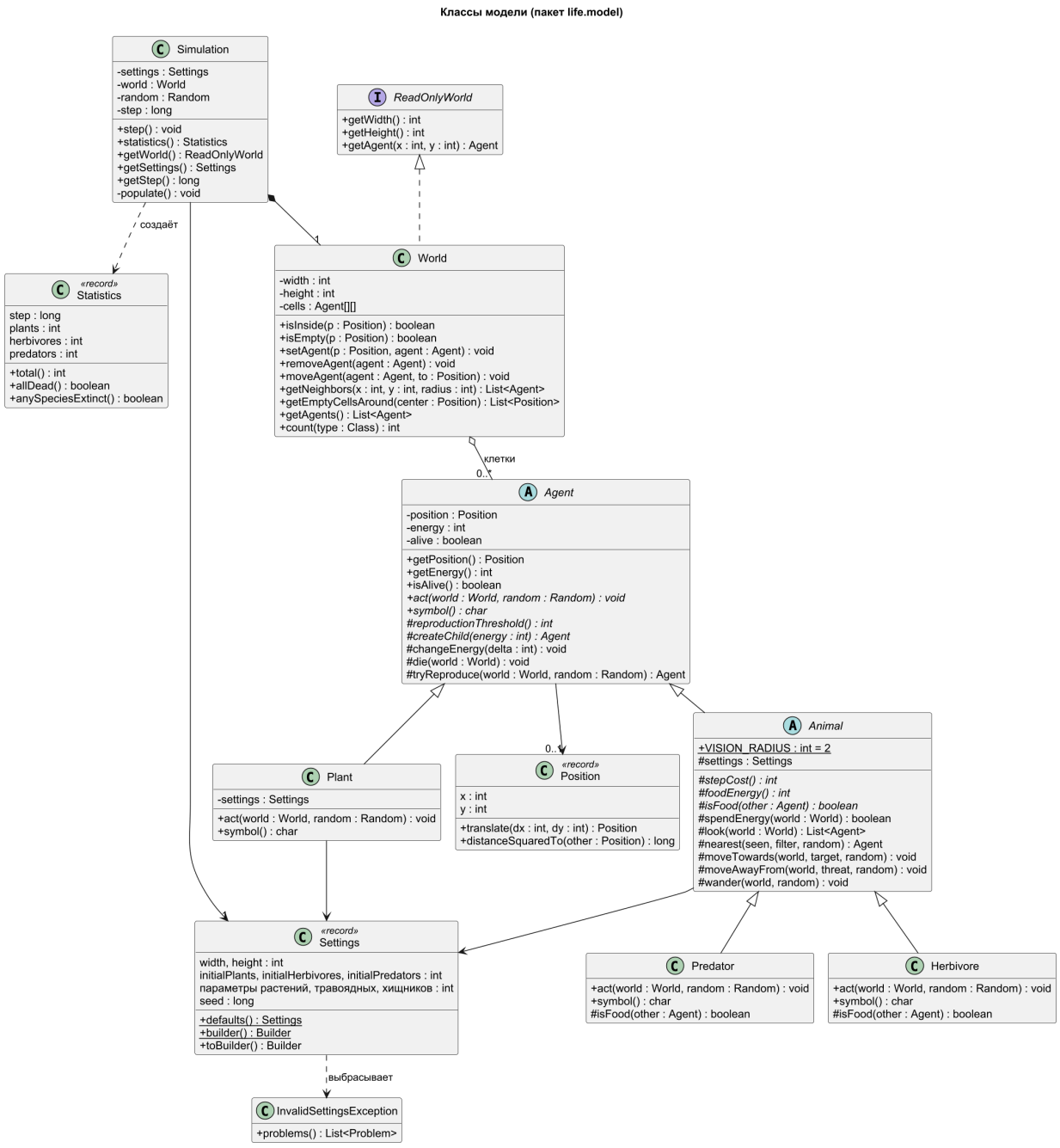


Рисунок 5 — Диаграмма классов модели (пакет life.model)

Классы управления и интерфейса показаны на рисунке 6. Оконный интерфейс (пакет life.view.gui) обращается к модели через контроллер SimulationController, консольная версия (пакет life.view.console) — напрямую к Simulation.

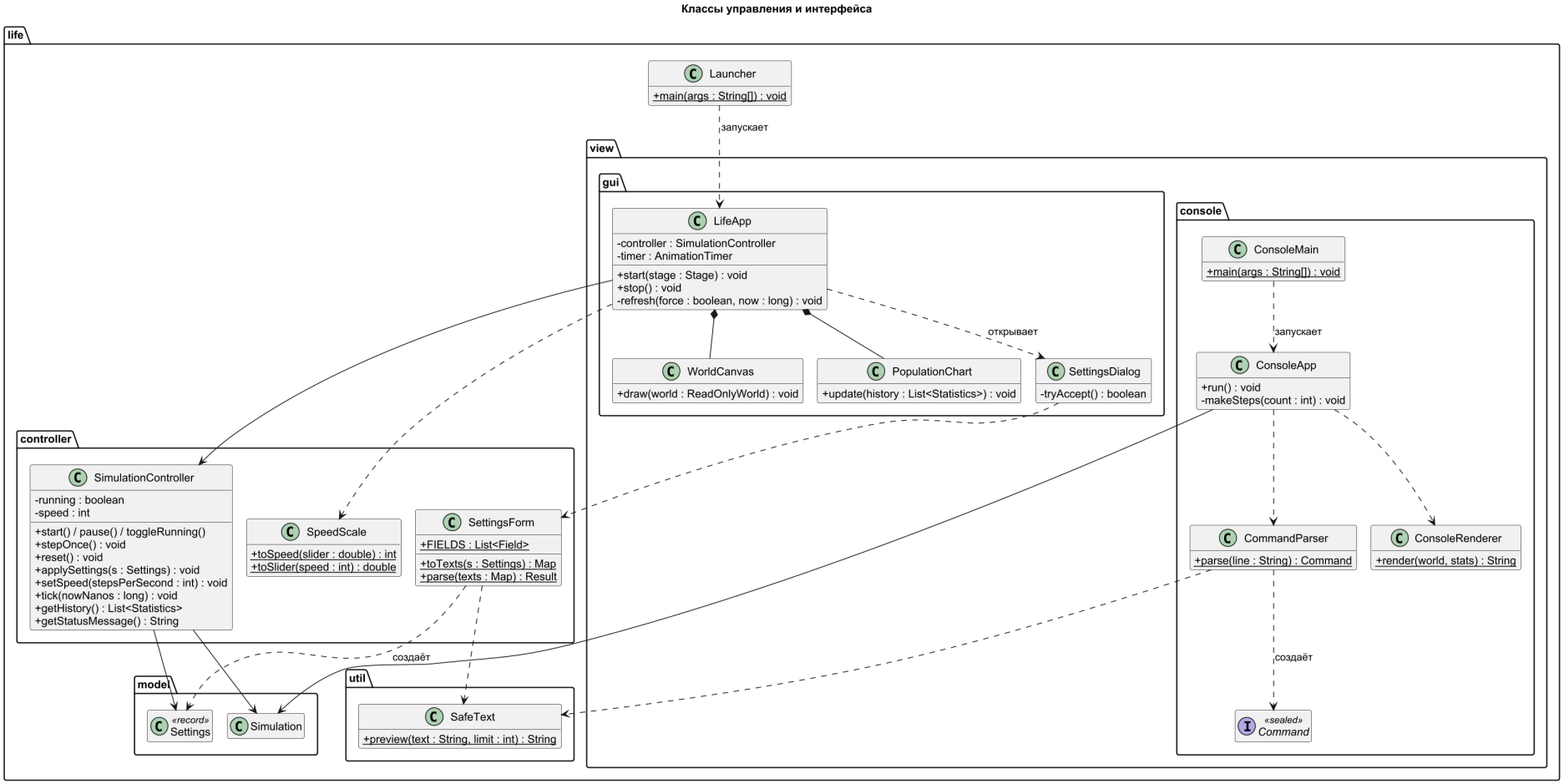


Рисунок 6 — Диаграмма классов управления и интерфейса

3.4 Диаграмма последовательности

На рисунке 7 показано взаимодействие объектов за один кадр окна: таймер вызывает окно, окно передаёт время контроллеру, контроллер выполняет нужное число шагов симуляции, симуляция вызывает ход каждого агента, агент запрашивает у поля содержимое области 5×5 и перемещается, после чего окно перерисовывает поле и график.

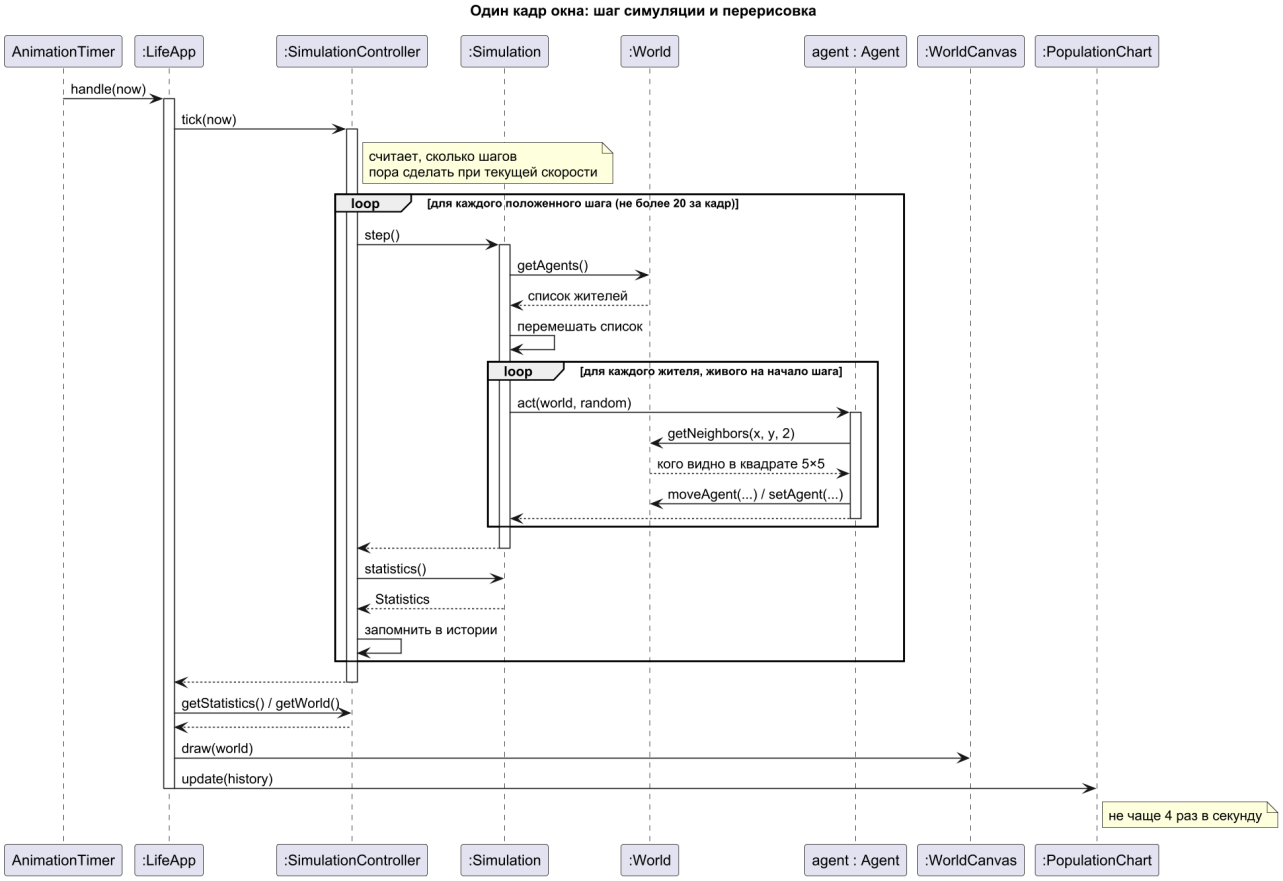


Рисунок 7 — Диаграмма последовательности одного кадра окна

3.5 Диаграмма компонентов

Программа оформлена как модуль Java life и разделена на пакеты по схеме «модель — представление — контроллер» (рисунок 8). Модель не зависит от интерфейса; оконная версия зависит от библиотеки JavaFX.

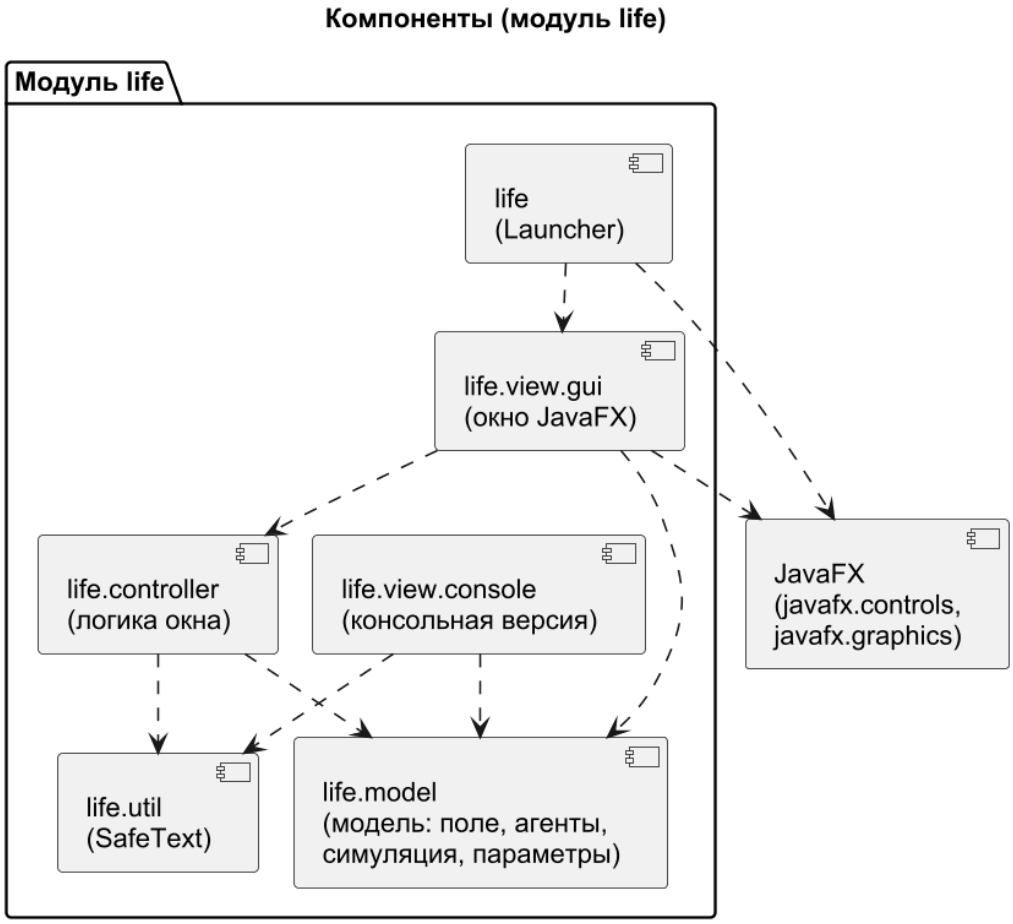


Рисунок 8 — Диаграмма компонентов

4 Реализация

4.1 Средства разработки

– язык Java 25 (JDK Temurin 25);
– библиотека пользовательского интерфейса JavaFX 25.0.4;
– система сборки Maven, среда разработки IntelliJ IDEA;
– библиотека тестирования JUnit 6.1.3;
– система контроля версий Git.

4.2 Структура программы

Таблица 3 — Пакеты программы

Пакет	Назначение
life	Launcher — точка входа оконной версии
life.model	Settings, World, ReadOnlyWorld, Position, Agent, Plant, Animal, Herbivore, Predator, Simulation, Statistics, InvalidSettingsException — модель
life.controller	SimulationController (старт, пауза, шаг, сброс, скорость), SettingsForm (разбор полей окна параметров), SpeedScale (шкала скорости)
life.view.console	ConsoleMain, ConsoleApp, CommandParser, Command, ConsoleRenderer — консольная версия
life.view.gui	LifeApp, WorldCanvas, PopulationChart, SettingsDialog — оконная версия
life.util	SafeText — безопасный вывод введённого пользователем текста в сообщениях

4.3 Применение принципов ООП

Таблица 4 — Принципы ООП в программе

Принцип	Реализация
Абстракция	Абстрактные классы Agent и Animal описывают «агента вообще» и «животное вообще»; создать их объекты нельзя, каждый вид обязан реализовать метод хода act()
Наследование	Agent → Plant; Agent → Animal → Herbivore, Predator. Общие данные и действия (энергия, деление, зрение, перемещение) реализованы один раз в предках
Полиморфизм	Simulation.step() вызывает agent.act(world, random) для переменной типа Agent; выполняется версия метода конкретного вида

Принцип	Реализация
Инкапсуляция	Поля энергии и позиции закрыты (private) и изменяются только методами; перемещать агентов может только World; интерфейс получает поле только для чтения (ReadOnlyWorld); Settings неизменяем и проверяет значения при создании

4.4 Консольная версия

Консольная версия выводит поле символами (P — растение, T — травоядное, X — хищник, «.» — пустая клетка) и строку со счётчиками, после чего читает команды пользователя (таблица 5). Любой некорректный ввод (буквы, дробные и отрицательные числа, слишком большие числа, управляющие символы) не приводит к аварийному завершению: программа выводит понятное сообщение и продолжает работу. При выполнении долгой команды раз в секунду печатается прогресс; если на поле не осталось агентов, шаги прекращаются с сообщением «Жизнь прекратилась».

Таблица 5 — Команды консольной версии

Ввод	Действие
Enter	один шаг
число N	N шагов подряд (от 1 до 100 000)
h, ?, помощь	справка
q, выход, конец ввода	выход из программы

4.5 Оконная версия

Главное окно (класс LifeApp) содержит поле в центре, панель управления справа и график численности снизу. Панель управления включает кнопки «Старт/Пауза», «Шаг», «Сброс», «Параметры…», слайдер скорости (от 1 до 500 шагов в секунду, логарифмическая шкала) и счётчики с цветовыми обозначениями видов.

Игровой цикл реализован таймером JavaFX AnimationTimer, который вызывается при каждой перерисовке экрана (около 60 раз в секунду). Контроллер накапливает прошедшее время, умноженное на скорость, и

выполняет столько целых шагов, сколько накопилось. За один кадр выполняется не более 20 шагов, поэтому окно не зависает; время, проведённое на паузе, не превращается в шаги. График обновляется не чаще четырёх раз в секунду и хранит последние 1000 точек.

Окно параметров содержит поля для всех 18 параметров, кнопки «Случайный seed» и «По умолчанию». Разбор введённых значений выполняет класс SettingsForm; при ошибках окно не закрывается, неверные поля подсвечиваются, под формой выводится список всех проблем.

4.6 Тестирование

Программа покрыта 417 автоматическими тестами JUnit, которые запускаются в случайном порядке (таблица 6). Тесты написаны до реализации соответствующего кода; для каждой проверки «действие выполнено» есть зеркальная проверка «лишнего не сделано». Логика кнопок проверена без открытия окна: время передаётся контроллеру вручную.

Таблица 6 — Автоматические тесты

Что проверяется	Тестов
Параметры и их проверка (Settings)	84
Поле, координаты, зрение (World, Position)	58
Агенты: энергия, деление, рост, поведение (Agent, Plant, Animal, Herbivore, Predator)	88
Шаг симуляции и статистика (Simulation, Statistics)	22
Сохранение всех видов на 5000 шагах при параметрах по умолчанию (BalanceTest)	5
Консольная версия: команды, вывод, цикл ввода	81
Контроллер окна, форма параметров, шкала скорости, SafeText	79
Всего	**417**

Качество тестов проверено мутационным тестированием: в код намеренно вносились небольшие ошибки (замена «>» на «>=», удаление вызова, перестановка аргументов и т. п.), после чего запускались тесты. Всего внесено 189 мутаций; мутации, не обнаруженные тестами с первого раза, указали на пропущенные случаи (например, квадратные поля не отличали

ширину от высоты), и для них были добавлены тесты. Оконная версия дополнительно проверена живым запуском: нажатием кнопок, вводом ошибочных параметров и закрытием окна.

5 Результаты работы

5.1 Консольная версия

На рисунке 9 показан запуск консольной версии: справка по командам, строка счётчиков и поле на шаге 0. На рисунке 10 — реакция на неизвестную команду «abc» и выполнение шага по Enter. На рисунке 11 — выполнение 10 шагов одной командой; видно быстрое размножение травоядных на первых шагах.



Рисунок 9 — Консольная версия: справка и начальное поле

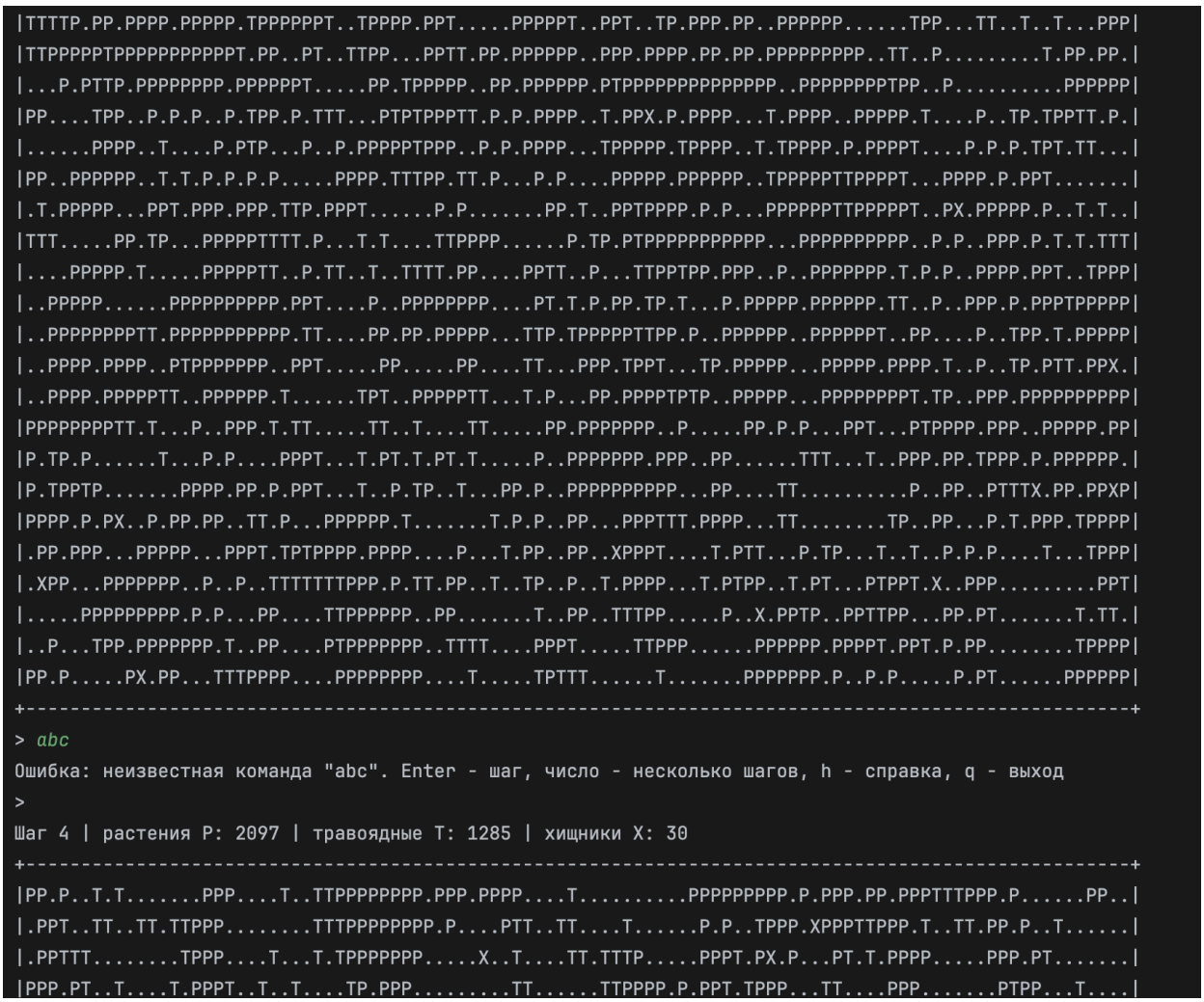


Рисунок 10 — Консольная версия: ошибка ввода и шаг

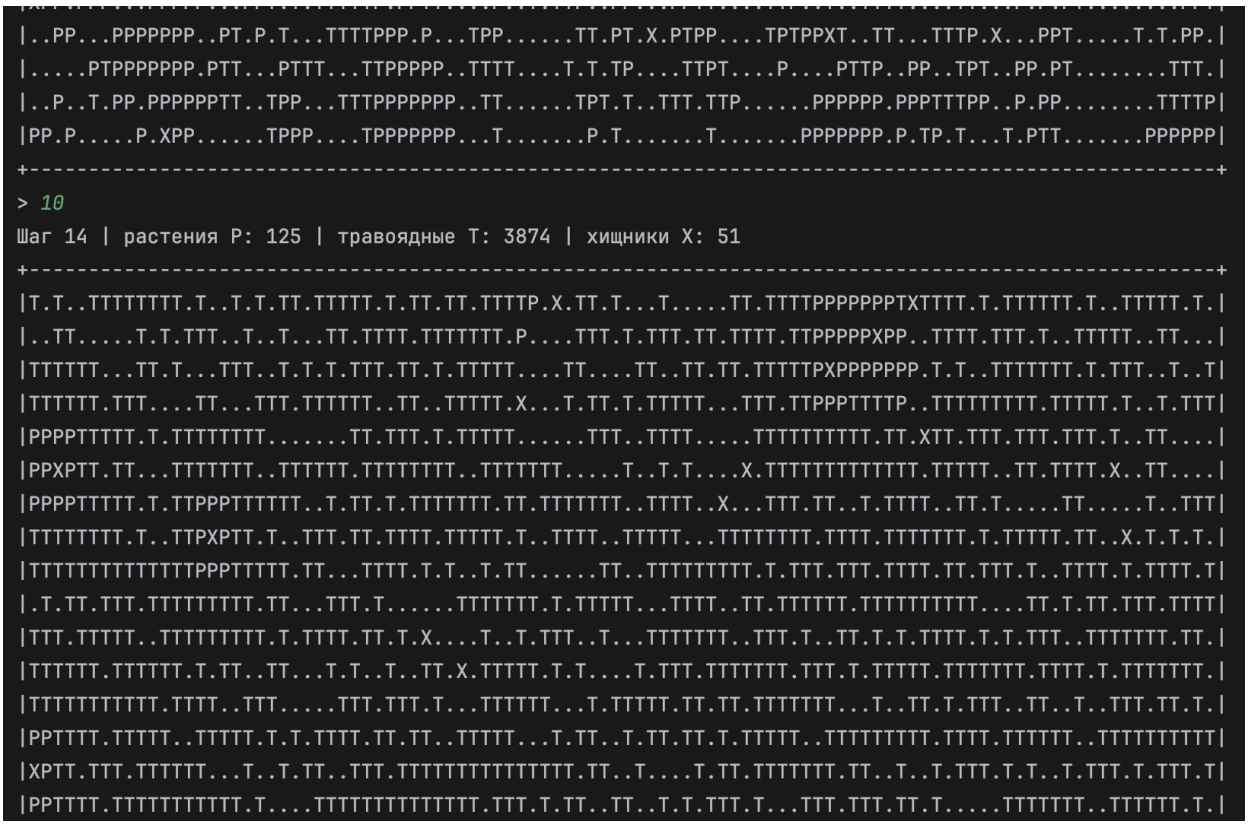


Рисунок 11 — Консольная версия: выполнение 10 шагов

5.2 Оконная версия

На рисунке 12 показано окно программы при запуске, на рисунке 13 — через 8 секунд работы (159 шагов). На графике видны колебания численности видов, при которых все три вида сохраняются. На рисунке 14 показано окно параметров после ввода ошибочных значений: окно не закрылось, неверные поля подсвечены, под формой перечислены все ошибки.

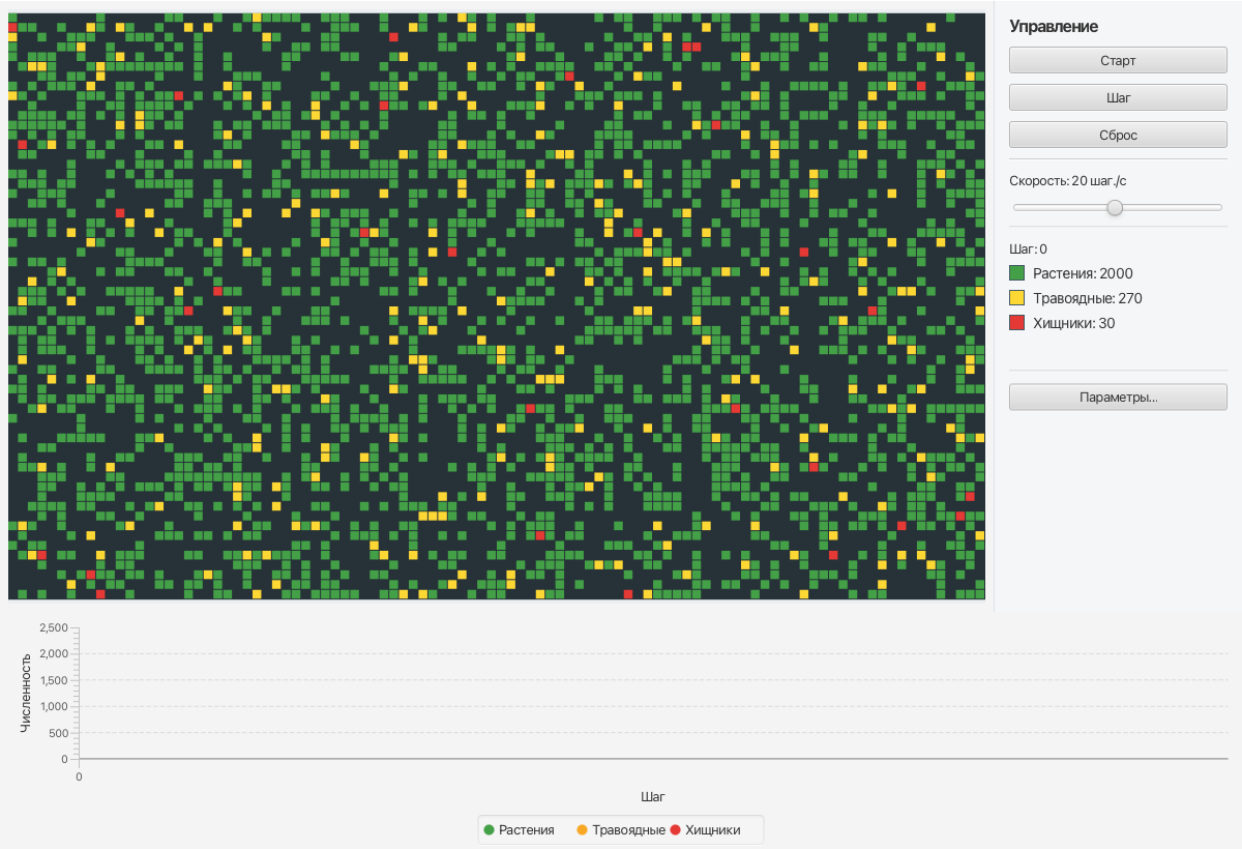


Рисунок 12 — Оконная версия при запуске

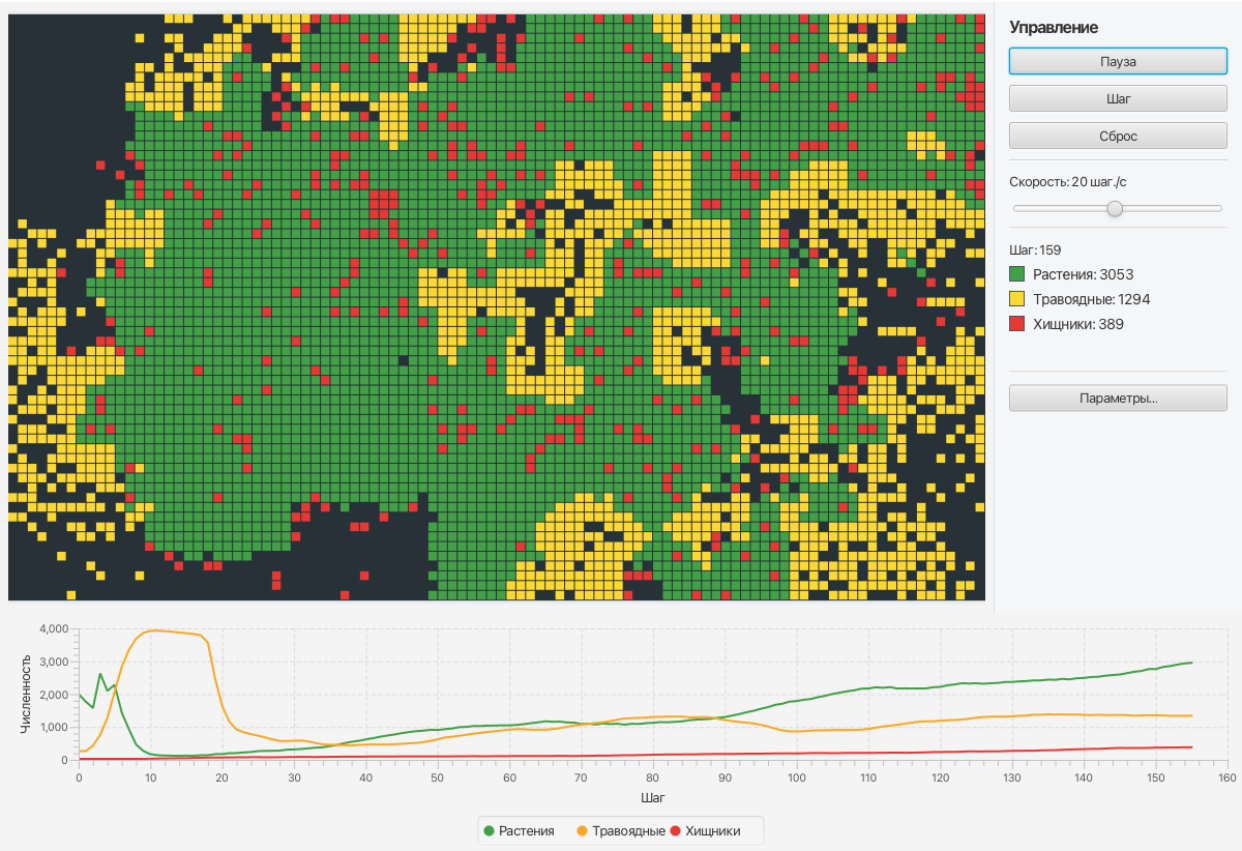


Рисунок 13 — Оконная версия через 8 секунд работы

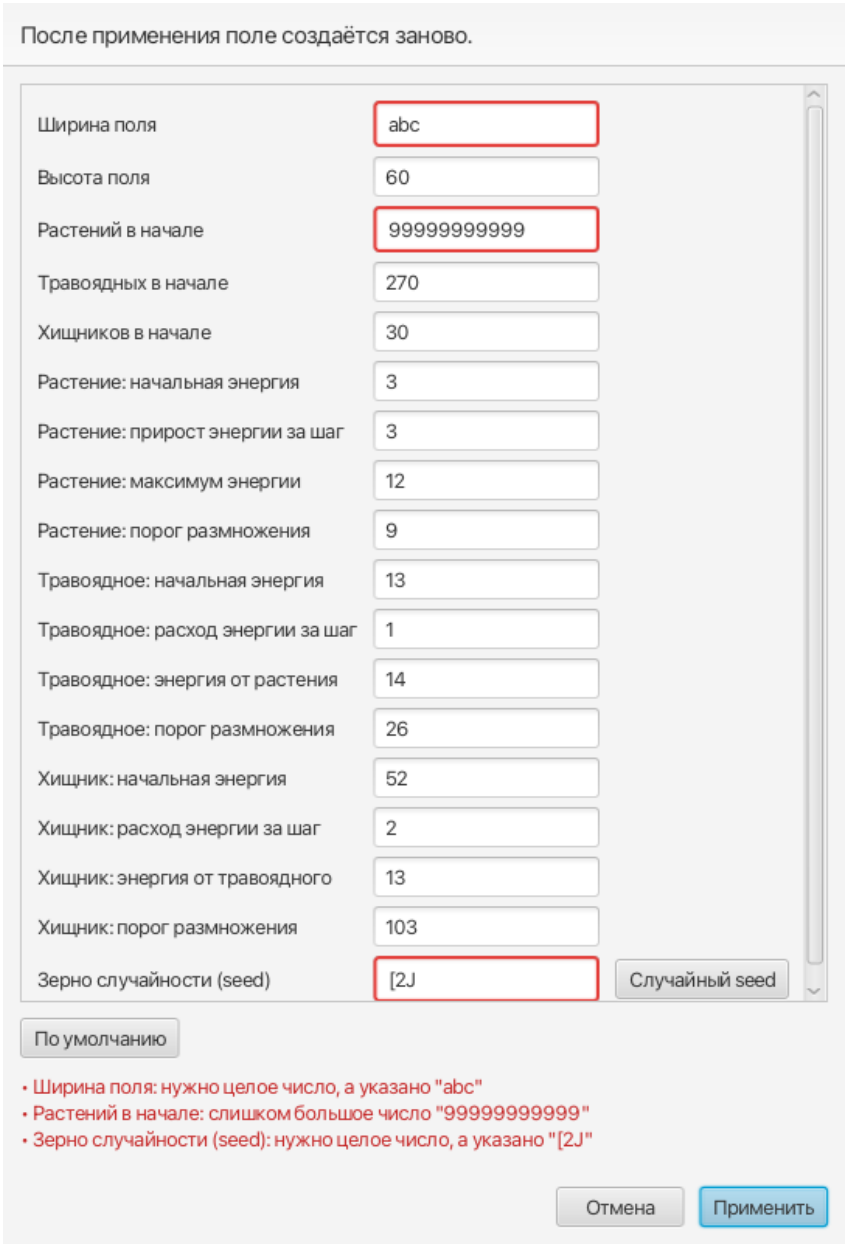


Рисунок 14 — Окно параметров с ошибочным вводом

Заключение

В ходе лабораторной работы разработана программа «Искусственная жизнь», моделирующая пищевую цепочку из растений, травоядных и хищников на клеточном поле. Агенты обладают зрением 5×5, действуют по логическим правилам, тратят и пополняют энергию, погибают и размножаются делением.

Программа построена на принципах ООП: абстрактные классы Agent и Animal задают общее поведение, конкретные виды наследуют его и реализуют собственную логику, симуляция работает с агентами полиморфно, данные агентов и параметры инкапсулированы. Код разделён на модель, контроллер и представление; реализованы консольная (предварительная) и оконная (окончательная) версии.

Параметры подобраны автоматическим перебором так, чтобы жизнь не прекращалась: при параметрах по умолчанию все три вида сохранились во всех 40 проверочных прогонах по 10 000 шагов. Корректность программы подтверждена 417 автоматическими тестами, мутационным тестированием и проверкой работы интерфейса.

Приложение А. Текст программы

Ниже приведён полный текст программы (без автоматических тестов). Файлы расположены в порядке: сборка и модуль, модель, контроллер, вспомогательный класс, консольная версия, оконная версия.

Файл pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>


    <groupId>ru.spbstu.oop</groupId>
    <artifactId>artificial-life</artifactId>
    <version>1.0</version>
    <name>Искусственная жизнь</name>


    <properties>
        <maven.compiler.release>25</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <javafx.version>25.0.4</javafx.version>
        <junit.version>6.1.3</junit.version>
    </properties>


    <dependencies>
        <dependency>
            <groupId>org.openjfx</groupId>
            <artifactId>javafx-controls</artifactId>
            <version>${javafx.version}</version>
        </dependency>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>${junit.version}</version>
            <scope>test</scope>
        </dependency>
    </dependencies>


    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.16.0</version>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.6.0</version>
            </plugin>
            <plugin>
                <groupId>org.openjfx</groupId>
                <artifactId>javafx-maven-plugin</artifactId>
                <version>0.0.8</version>
                <configuration>
                    <!-- модуль/класс: программа оформлена как модуль Java -->
                    <mainClass>life/life.Launcher</mainClass>
                    <options>
                        <!-- разрешить JavaFX загружать системный код без предупреждения --
>
                        <option>--enable-native-access=javafx.graphics</option>
                    </options>
```

```
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

Файл src/main/java/module-info.java

```
/**
 * Модуль программы «Искусственная жизнь»: какие пакеты есть и что им нужно.
 * JavaFX требует, чтобы приложение было модулем, иначе при запуске выводится
предупреждение.
 */
module life {
    requires javafx.controls;


    exports life;
    exports life.model;
    exports life.controller;
    exports life.util;
    exports life.view.console;
    // JavaFX сам создаёт объект окна LifeApp и читает стили из этого пакета
    exports life.view.gui to javafx.graphics;
    opens life.view.gui to javafx.graphics;
}
```

Файл src/main/java/life/Launcher.java

```
package life;


import javafx.application.Application;
import life.view.gui.LifeApp;


/**
 * Точка входа оконной версии.
 * Отдельный класс, а не сам LifeApp: если главный класс наследует Application,
 * запуск без модулей JavaFX падает с ошибкой «JavaFX runtime components are missing».
 */
public final class Launcher {


    private Launcher() {
    }


    public static void main(String[] args) {
        Application.launch(LifeApp.class, args);
    }
}
```

Файл src/main/java/life/model/Settings.java

```
package life.model;


import java.util.ArrayList;
import java.util.List;


/**
 * Все параметры симуляции в одном месте.
 * Объект неизменяемый: создаётся через {@link Builder} и проверяется при создании,
 * поэтому некорректные настройки не могут попасть в модель.
 */
public record Settings(
        int width,
        int height,
        int initialPlants,
        int initialHerbivores,
        int initialPredators,
        int plantStartEnergy,
        int plantGrowth,
```

```
        int plantMaxEnergy,
        int plantReproductionThreshold,
        int herbivoreStartEnergy,
        int herbivoreStepCost,
        int herbivoreFoodEnergy,
        int herbivoreReproductionThreshold,
        int predatorStartEnergy,
        int predatorStepCost,
        int predatorFoodEnergy,
        int predatorReproductionThreshold,
        long seed) {


    public static final int MAX_SIZE = 1000;
    public static final int MAX_ENERGY = 1_000_000;


    public Settings {
        Checker check = new Checker();


        boolean sizeOk = check.range("width", "Ширина поля", width, 1, MAX_SIZE)
                & check.range("height", "Высота поля", height, 1, MAX_SIZE);
        boolean countsOk = check.range("initialPlants", "Начальное число растений",
initialPlants, 0, Integer.MAX_VALUE)
                & check.range("initialHerbivores", "Начальное число травоядных",
initialHerbivores, 0, Integer.MAX_VALUE)
                & check.range("initialPredators", "Начальное число хищников",
initialPredators, 0, Integer.MAX_VALUE);
        if (sizeOk && countsOk) {
            // long, чтобы сумма огромных чисел не переполнилась и не превратилась в «мало»
            long agents = (long) initialPlants + initialHerbivores + initialPredators;
            long cells = (long) width * height;
            if (agents > cells) {
                check.fail("initialCount", "Жителей " + agents + ", а клеток на поле только
" + cells);
            }
        }


        check.range("plantStartEnergy", "Начальная энергия растения", plantStartEnergy, 1,
MAX_ENERGY);
        check.range("plantGrowth", "Прирост энергии растения за шаг", plantGrowth, 1,
MAX_ENERGY);
        check.range("plantMaxEnergy", "Максимум энергии растения", plantMaxEnergy, 1,
MAX_ENERGY);
        check.range("plantReproductionThreshold", "Порог размножения растения",
plantReproductionThreshold, 1, MAX_ENERGY);
        if (plantStartEnergy > plantMaxEnergy) {
            check.fail("plantStartEnergy", "Начальная энергия растения (" +
plantStartEnergy
                    + ") больше его максимума (" + plantMaxEnergy + ")");
        }
        if (plantReproductionThreshold >= plantMaxEnergy) {
            check.fail("plantReproductionThreshold", "Порог размножения растения (" +
plantReproductionThreshold
                    + ") должен быть меньше максимума энергии (" + plantMaxEnergy + "),
иначе растения не размножатся");
        }


        check.range("herbivoreStartEnergy", "Начальная энергия травоядного",
herbivoreStartEnergy, 1, MAX_ENERGY);
        check.range("herbivoreStepCost", "Расход энергии травоядного за шаг",
herbivoreStepCost, 1, MAX_ENERGY);
        check.range("herbivoreFoodEnergy", "Энергия травоядного от растения",
herbivoreFoodEnergy, 1, MAX_ENERGY);
        check.range("herbivoreReproductionThreshold", "Порог размножения травоядного",
herbivoreReproductionThreshold, 1, MAX_ENERGY);


        check.range("predatorStartEnergy", "Начальная энергия хищника",
predatorStartEnergy, 1, MAX_ENERGY);
        check.range("predatorStepCost", "Расход энергии хищника за шаг", predatorStepCost,
1, MAX_ENERGY);
        check.range("predatorFoodEnergy", "Энергия хищника от травоядного",
predatorFoodEnergy, 1, MAX_ENERGY);
```

```
        check.range("predatorReproductionThreshold", "Порог размножения хищника",
predatorReproductionThreshold, 1, MAX_ENERGY);

        check.throwIfFailed();
    }

    /** Копит найденные проблемы, чтобы сообщить обо всех сразу. */
    private static final class Checker {
        private final List<InvalidSettingsException.Problem> problems = new ArrayList<>();

        /** Проверяет, что значение в [min, max]. Возвращает true, если всё в порядке. */
        boolean range(String field, String title, int value, int min, int max) {
            if (value >= min && value <= max) {
                return true;
            }
            String allowed = max == Integer.MAX_VALUE ? "не меньше " + min : "от " + min +
" до " + max;
            fail(field, title + ": допустимо " + allowed + ", указано " + value);
            return false;
        }

        void fail(String field, String message) {
            problems.add(new InvalidSettingsException.Problem(field, message));
        }

        void throwIfFailed() {
            if (!problems.isEmpty()) {
                throw new InvalidSettingsException(problems);
            }
        }
    }

    public static Settings defaults() {
        return builder().build();
    }

    public static Builder builder() {
        return new Builder();
    }

    /** Билдер с текущими значениями — удобно поменять пару полей у готовых настроек. */
    public Builder toBuilder() {
        return new Builder()
                .width(width).height(height)

.initialPlants(initialPlants).initialHerbivores(initialHerbivores).initialPredators(initial
Predators)
                .plantStartEnergy(plantStartEnergy).plantGrowth(plantGrowth)

.plantMaxEnergy(plantMaxEnergy).plantReproductionThreshold(plantReproductionThreshold)

.herbivoreStartEnergy(herbivoreStartEnergy).herbivoreStepCost(herbivoreStepCost)

.herbivoreFoodEnergy(herbivoreFoodEnergy).herbivoreReproductionThreshold(herbivoreReproduct
ionThreshold)

.predatorStartEnergy(predatorStartEnergy).predatorStepCost(predatorStepCost)

.predatorFoodEnergy(predatorFoodEnergy).predatorReproductionThreshold(predatorReproductionT
hreshold)
                .seed(seed);
    }

    /**
     * Пошаговая сборка настроек: меняем только нужные поля, остальные берутся по
умолчанию.
     * Умолчания подобраны перебором так, чтобы все три вида жили долго (см. BalanceTest).
     */
    public static final class Builder {
```

```
        private int width = 100;
        private int height = 60;
        private int initialPlants = 2000;
        private int initialHerbivores = 270;
        private int initialPredators = 30;
        private int plantStartEnergy = 3;
        private int plantGrowth = 3;
        private int plantMaxEnergy = 12;
        private int plantReproductionThreshold = 9;
        private int herbivoreStartEnergy = 13;
        private int herbivoreStepCost = 1;
        private int herbivoreFoodEnergy = 14;
        private int herbivoreReproductionThreshold = 26;
        private int predatorStartEnergy = 52;
        private int predatorStepCost = 2;
        private int predatorFoodEnergy = 13;
        private int predatorReproductionThreshold = 103;
        private long seed = 42;


        private Builder() {
        }


        public Builder width(int value) { width = value; return this; }
        public Builder height(int value) { height = value; return this; }
        public Builder initialPlants(int value) { initialPlants = value; return this; }
        public Builder initialHerbivores(int value) { initialHerbivores = value; return
this; }
        public Builder initialPredators(int value) { initialPredators = value; return this;
}
        public Builder plantStartEnergy(int value) { plantStartEnergy = value; return this;
}
        public Builder plantGrowth(int value) { plantGrowth = value; return this; }
        public Builder plantMaxEnergy(int value) { plantMaxEnergy = value; return this; }
        public Builder plantReproductionThreshold(int value) { plantReproductionThreshold =
value; return this; }
        public Builder herbivoreStartEnergy(int value) { herbivoreStartEnergy = value;
return this; }
        public Builder herbivoreStepCost(int value) { herbivoreStepCost = value; return
this; }
        public Builder herbivoreFoodEnergy(int value) { herbivoreFoodEnergy = value; return
this; }
        public Builder herbivoreReproductionThreshold(int value) {
herbivoreReproductionThreshold = value; return this; }
        public Builder predatorStartEnergy(int value) { predatorStartEnergy = value; return
this; }
        public Builder predatorStepCost(int value) { predatorStepCost = value; return this;
}
        public Builder predatorFoodEnergy(int value) { predatorFoodEnergy = value; return
this; }
        public Builder predatorReproductionThreshold(int value) {
predatorReproductionThreshold = value; return this; }
        public Builder seed(long value) { seed = value; return this; }


        public Settings build() {
            return new Settings(width, height,
                    initialPlants, initialHerbivores, initialPredators,
                    plantStartEnergy, plantGrowth, plantMaxEnergy,
plantReproductionThreshold,
                    herbivoreStartEnergy, herbivoreStepCost, herbivoreFoodEnergy,
herbivoreReproductionThreshold,
                    predatorStartEnergy, predatorStepCost, predatorFoodEnergy,
predatorReproductionThreshold,
                    seed);
        }
    }
}
```

Файл src/main/java/life/model/InvalidSettingsException.java

```
package life.model;


import java.util.List;
```

```
/**
 * Настройки симуляции некорректны. Содержит сразу все найденные проблемы,
 * чтобы пользователь исправил их за один раз, а не по одной.
 */
public class InvalidSettingsException extends IllegalArgumentException {

    /** Одна проблема: какое поле и что с ним не так (понятным языком). */
    public record Problem(String field, String message) {
    }

    private final List<Problem> problems;

    public InvalidSettingsException(List<Problem> problems) {
        super("Некорректные настройки: " + String.join("; ",
problems.stream().map(Problem::message).toList()));
        this.problems = List.copyOf(problems);
    }

    public List<Problem> problems() {
        return problems;
    }
}
```

Файл src/main/java/life/model/Position.java

```
package life.model;

/** Координаты клетки на поле: x — столбец (слева направо), y — строка (сверху вниз). */
public record Position(int x, int y) {

    /** Соседняя клетка, сдвинутая на (dx, dy). */
    public Position translate(int dx, int dy) {
        return new Position(x + dx, y + dy);
    }

    /** Квадрат расстояния до другой клетки. Корень не нужен: сравнивать «ближе/дальше»
можно и так. */
    public long distanceSquaredTo(Position other) {
        long dx = (long) x - other.x;
        long dy = (long) y - other.y;
        return dx * dx + dy * dy;
    }
}
```

Файл src/main/java/life/model/ReadOnlyWorld.java

```
package life.model;

/** Поле «только для чтения»: экран может его рисовать, но не может ничего на нём менять.
*/
public interface ReadOnlyWorld {

    int getWidth();

    int getHeight();

    /** Житель в клетке или null, если клетка пустая. */
    Agent getAgent(int x, int y);
}
```

Файл src/main/java/life/model/World.java

```
package life.model;
```

```
import java.util.ArrayList;
import java.util.List;


/**
 * Поле — двумерная сетка клеток. В клетке либо пусто, либо ровно один житель.
 * Только поле двигает жителей, поэтому координаты в поле и в самом жителе всегда
совпадают.
 */
public class World implements ReadOnlyWorld {


    private final int width;
    private final int height;
    /** cells[y][x]: сначала строка, потом столбец. */
    private final Agent[][] cells;


    public World(int width, int height) {
        if (width < 1 || height < 1) {
            throw new IllegalArgumentException("Размер поля должен быть положительным,
указано " + width + "×" + height);
        }
        this.width = width;
        this.height = height;
        this.cells = new Agent[height][width];
    }


    public int getWidth() {
        return width;
    }


    public int getHeight() {
        return height;
    }


    public boolean isInside(Position p) {
        return p.x() >= 0 && p.x() < width && p.y() >= 0 && p.y() < height;
    }


    public Agent getAgent(int x, int y) {
        return getAgent(new Position(x, y));
    }


    /** Житель в клетке или null, если клетка пустая. */
    public Agent getAgent(Position p) {
        requireInside(p);
        return cells[p.y()][p.x()];
    }


    /** Клетка на поле и в ней никого нет. Клетка за краем поля пустой не считается. */
    public boolean isEmpty(Position p) {
        return isInside(p) && cells[p.y()][p.x()] == null;
    }


    public void setAgent(int x, int y, Agent agent) {
        setAgent(new Position(x, y), agent);
    }


    /**
     * Размещает жителя в пустой клетке. Если agent == null — очищает клетку.
     *
     * @throws IllegalArgumentException клетка за краем поля
     * @throws IllegalStateException    клетка занята или житель уже стоит на каком-то поле
     */
    public void setAgent(Position p, Agent agent) {
        requireInside(p);
        Agent current = cells[p.y()][p.x()];
        if (agent == null) {
```

```
            if (current != null) {
                current.setPosition(null);
                cells[p.y()][p.x()] = null;
            }
            return;
        }
        if (agent.getPosition() != null) {
            throw new IllegalStateException("Житель уже стоит на поле в клетке " +
agent.getPosition());
        }
        if (current != null) {
            throw new IllegalStateException("Клетка " + p + " уже занята");
        }
        cells[p.y()][p.x()] = agent;
        agent.setPosition(p);
    }


    public void removeAgent(Agent agent) {
        requireOnThisWorld(agent);
        setAgent(agent.getPosition(), null);
    }


    /** Переносит жителя в пустую клетку. */
    public void moveAgent(Agent agent, Position to) {
        requireOnThisWorld(agent);
        requireInside(to);
        if (cells[to.y()][to.x()] != null) {
            throw new IllegalStateException("Клетка " + to + " уже занята");
        }
        Position from = agent.getPosition();
        cells[from.y()][from.x()] = null;
        cells[to.y()][to.x()] = agent;
        agent.setPosition(to);
    }


    /**
     * Все жители в квадрате со стороной 2·radius+1 вокруг клетки (x, y), кроме того, кто в
самой клетке.
     * Радиус 2 — это зрение 5×5. Часть квадрата за краем поля просто не рассматривается.
     */
    public List<Agent> getNeighbors(int x, int y, int radius) {
        if (radius < 0) {
            throw new IllegalArgumentException("Радиус не может быть отрицательным: " +
radius);
        }
        requireInside(new Position(x, y));
        // long, чтобы x + огромный радиус не переполнился
        int top = (int) Math.max(0, (long) y - radius);
        int bottom = (int) Math.min(height - 1, (long) y + radius);
        int left = (int) Math.max(0, (long) x - radius);
        int right = (int) Math.min(width - 1, (long) x + radius);


        List<Agent> result = new ArrayList<>();
        for (int row = top; row <= bottom; row++) {
            for (int col = left; col <= right; col++) {
                Agent agent = cells[row][col];
                if (agent != null && (row != y || col != x)) {
                    result.add(agent);
                }
            }
        }
        return List.copyOf(result);
    }


    /** Свободные клетки среди восьми соседних (по сторонам и по диагонали). */
    public List<Position> getEmptyCellsAround(Position center) {
        requireInside(center);
        List<Position> result = new ArrayList<>();
        for (int dy = -1; dy <= 1; dy++) {
            for (int dx = -1; dx <= 1; dx++) {
                Position p = center.translate(dx, dy);
                if ((dx != 0 || dy != 0) && isEmpty(p)) {
```

```
                    result.add(p);
                }
            }
        }
        return List.copyOf(result);
    }

    /** Все жители поля, построчно сверху вниз. Список — копия: менять поле через него
нельзя. */
    public List<Agent> getAgents() {
        List<Agent> result = new ArrayList<>();
        for (Agent[] row : cells) {
            for (Agent agent : row) {
                if (agent != null) {
                    result.add(agent);
                }
            }
        }
        return List.copyOf(result);
    }

    /** Сколько на поле жителей данного типа (включая наследников). */
    public int count(Class<? extends Agent> type) {
        int count = 0;
        for (Agent[] row : cells) {
            for (Agent agent : row) {
                if (type.isInstance(agent)) {
                    count++;
                }
            }
        }
        return count;
    }

    private void requireInside(Position p) {
        if (!isInside(p)) {
            throw new IllegalArgumentException("Клетка " + p + " за краем поля " + width +
"×" + height);
        }
    }

    private void requireOnThisWorld(Agent agent) {
        Position p = agent.getPosition();
        if (p == null || !isInside(p) || cells[p.y()][p.x()] != agent) {
            throw new IllegalStateException("Этого жителя нет на поле");
        }
    }
}
```

Файл src/main/java/life/model/Agent.java

```
package life.model;

import java.util.List;
import java.util.Random;

/**
 * Житель поля: растение или животное.
 * Хранит общие для всех данные — где стоит, сколько энергии, жив ли —
 * и общие действия: изменение энергии, гибель, размножение делением.
 * Что именно житель делает за ход, решает каждый наследник сам (метод {@link #act}).
 */
public abstract class Agent {

    /** Клетка, где стоит житель; null — его нет на поле. Меняет только {@link World}. */
    private Position position;
    private int energy;
    private boolean alive = true;
```

```
    protected Agent(int energy) {
        if (energy < 1) {
            throw new IllegalArgumentException("Житель должен рождаться с положительной
энергией, указано " + energy);
        }
        this.energy = energy;
    }


    public Position getPosition() {
        return position;
    }


    public int getEnergy() {
        return energy;
    }


    public boolean isAlive() {
        return alive;
    }


    /** Один ход жителя. */
    public abstract void act(World world, Random random);


    /** Буква для консоли: P — растение, T — травоядное, X — хищник. */
    public abstract char symbol();


    /** Порог размножения: при энергии строго больше порога житель делится. */
    protected abstract int reproductionThreshold();


    /** Новый житель того же вида с заданной энергией. */
    protected abstract Agent createChild(int energy);


    /** Прибавляет (или отнимает) энергию. Энергия не бывает меньше нуля и не
переполняется. */
    protected void changeEnergy(int delta) {
        energy = Math.clamp((long) energy + delta, 0, Integer.MAX_VALUE);
    }


    /** Житель погибает и исчезает с поля. Повторный вызов ничего не делает. */
    protected void die(World world) {
        alive = false;
        if (position != null) {
            world.removeAgent(this);
        }
    }


    /**
     * Размножение делением: если энергии больше порога и рядом есть свободная клетка,
     * в случайной свободной соседней клетке появляется ребёнок с половиной энергии
родителя.
     *
     * @return ребёнок или null, если размножиться не получилось
     */
    protected Agent tryReproduce(World world, Random random) {
        if (!alive || energy <= reproductionThreshold()) {
            return null;
        }
        List<Position> free = world.getEmptyCellsAround(position);
        if (free.isEmpty()) {
            return null;
        }
        Position place = free.get(random.nextInt(free.size()));
        Agent child = createChild(energy / 2);
        energy -= child.getEnergy();
        world.setAgent(place, child);
        return child;
    }
```

```
    void setPosition(Position position) {
        this.position = position;
    }
}
```

Файл src/main/java/life/model/Plant.java

```
package life.model;


import java.util.Random;


/** Растение: не двигается, само набирает энергию «от солнца» и разрастается в соседние
клетки. */
public class Plant extends Agent {


    private final Settings settings;


    public Plant(Settings settings) {
        this(settings, settings.plantStartEnergy());
    }


    public Plant(Settings settings, int energy) {
        super(energy);
        if (energy > settings.plantMaxEnergy()) {
            throw new IllegalArgumentException("Энергия растения " + energy
                    + " больше максимума " + settings.plantMaxEnergy());
        }
        this.settings = settings;
    }


    /** Ход растения: подрасти (не выше максимума), затем размножиться, если энергии
хватает. */
    @Override
    public void act(World world, Random random) {
        if (!isAlive()) {
            return;
        }
        int room = settings.plantMaxEnergy() - getEnergy();
        changeEnergy(Math.min(settings.plantGrowth(), room));
        tryReproduce(world, random);
    }


    @Override
    public char symbol() {
        return 'P';
    }


    @Override
    protected int reproductionThreshold() {
        return settings.plantReproductionThreshold();
    }


    @Override
    protected Agent createChild(int energy) {
        return new Plant(settings, energy);
    }
}
```

Файл src/main/java/life/model/Animal.java

```
package life.model;


import java.util.ArrayList;
import java.util.List;
import java.util.Random;
import java.util.function.Predicate;
import java.util.function.ToLongFunction;
```

```
/**
 * Животное: видит квадрат 5×5 вокруг себя, ходит на одну клетку за ход и тратит энергию.
 * Здесь собраны общие для всех животных умения; какую еду искать и кого бояться,
 * решают наследники в своём методе {@link #act}.
 */
public abstract class Animal extends Agent {


    /** Радиус зрения: 2 клетки в каждую сторону — квадрат 5×5 с животным в центре. */
    public static final int VISION_RADIUS = 2;


    protected final Settings settings;


    protected Animal(Settings settings, int energy) {
        super(energy);
        this.settings = settings;
    }


    /** Сколько энергии тратится за ход. */
    protected abstract int stepCost();


    /** Сколько энергии даёт одна съеденная еда. */
    protected abstract int foodEnergy();


    /** Можно ли это съесть. */
    protected abstract boolean isFood(Agent other);


    /**
     * Тратит энергию за ход. Если энергия кончилась — животное погибает.
     *
     * @return true, если животное выжило
     */
    protected boolean spendEnergy(World world) {
        changeEnergy(-stepCost());
        if (getEnergy() == 0) {
            die(world);
            return false;
        }
        return true;
    }


    /** Все, кого животное видит в квадрате 5×5 вокруг себя. */
    protected List<Agent> look(World world) {
        Position here = getPosition();
        return world.getNeighbors(here.x(), here.y(), VISION_RADIUS);
    }


    /** Ближайший из увиденных, кто подходит под условие; при равном расстоянии —
случайный. null — таких нет. */
    protected Agent nearest(List<Agent> seen, Predicate<Agent> filter, Random random) {
        List<Agent> closest = new ArrayList<>();
        long bestDistance = Long.MAX_VALUE;
        for (Agent other : seen) {
            if (!filter.test(other)) {
                continue;
            }
            long distance = getPosition().distanceSquaredTo(other.getPosition());
            if (distance < bestDistance) {
                closest.clear();
                bestDistance = distance;
            }
            if (distance == bestDistance) {
                closest.add(other);
            }
        }
        return closest.isEmpty() ? null : closest.get(random.nextInt(closest.size()));
    }
```

```
    /** Шаг к цели. Если на выбранной клетке еда — съедает её. Если ближе подойти нельзя —
стоит. */
    protected void moveTowards(World world, Position target, Random random) {
        Position step = chooseStep(world, true, cell -> cell.distanceSquaredTo(target),
random);
        if (step != null) {
            stepTo(world, step);
        }
    }


    /** Шаг в свободную клетку, самую дальнюю от угрозы. Если дальше уйти нельзя — стоит.
*/
    protected void moveAwayFrom(World world, Position threat, Random random) {
        Position step = chooseStep(world, false, cell -> -cell.distanceSquaredTo(threat),
random);
        if (step != null) {
            stepTo(world, step);
        }
    }


    /** Шаг в случайную свободную соседнюю клетку. Если все заняты — стоит. */
    protected void wander(World world, Random random) {
        List<Position> free = world.getEmptyCellsAround(getPosition());
        if (!free.isEmpty()) {
            world.moveAgent(this, free.get(random.nextInt(free.size())));
        }
    }


    /**
     * Выбирает соседнюю клетку с наименьшей «оценкой» — но только если она лучше, чем
остаться на месте.
     * Проходимы пустые клетки, а если mayEat — ещё и клетки с едой.
     *
     * @return клетка для шага или null — лучше стоять
     */
    private Position chooseStep(World world, boolean mayEat, ToLongFunction<Position>
score, Random random) {
        Position here = getPosition();
        long stayScore = score.applyAsLong(here);
        long bestScore = stayScore;
        List<Position> bestCells = new ArrayList<>();
        for (int dy = -1; dy <= 1; dy++) {
            for (int dx = -1; dx <= 1; dx++) {
                Position cell = here.translate(dx, dy);
                if ((dx == 0 && dy == 0) || !world.isInside(cell)) {
                    continue;
                }
                Agent occupant = world.getAgent(cell);
                if (occupant != null && !(mayEat && isFood(occupant))) {
                    continue;
                }
                long cellScore = score.applyAsLong(cell);
                if (cellScore < bestScore) {
                    bestCells.clear();
                    bestScore = cellScore;
                }
                if (cellScore == bestScore && bestScore < stayScore) {
                    bestCells.add(cell);
                }
            }
        }
        return bestCells.isEmpty() ? null :
bestCells.get(random.nextInt(bestCells.size()));
    }


    /** Переходит в клетку; если там еда — сначала съедает её. */
    private void stepTo(World world, Position cell) {
        Agent food = world.getAgent(cell);
        if (food != null) {
            changeEnergy(foodEnergy());
            food.die(world);
        }
```

```
        world.moveAgent(this, cell);
    }
}
```

Файл src/main/java/life/model/Herbivore.java

```
package life.model;


import java.util.List;
import java.util.Random;


/** Травоядное: ест растения и убегает от хищников. */
public class Herbivore extends Animal {


    public Herbivore(Settings settings) {
        this(settings, settings.herbivoreStartEnergy());
    }


    public Herbivore(Settings settings, int energy) {
        super(settings, energy);
    }


    /**
     * Ход травоядного:
     * ЕСЛИ видит хищника — убегает в противоположную сторону;
     * ИНАЧЕ ЕСЛИ видит растение — идёт к нему (и съедает, если дошло);
     * ИНАЧЕ бродит случайно.
     * Перед этим тратит энергию, после — размножается, если энергии хватает.
     */
    @Override
    public void act(World world, Random random) {
        if (!isAlive() || !spendEnergy(world)) {
            return;
        }
        List<Agent> seen = look(world);
        Agent predator = nearest(seen, other -> other instanceof Predator, random);
        if (predator != null) {
            moveAwayFrom(world, predator.getPosition(), random);
        } else {
            Agent plant = nearest(seen, this::isFood, random);
            if (plant != null) {
                moveTowards(world, plant.getPosition(), random);
            } else {
                wander(world, random);
            }
        }
        tryReproduce(world, random);
    }


    @Override
    public char symbol() {
        return 'T';
    }


    @Override
    protected int stepCost() {
        return settings.herbivoreStepCost();
    }


    @Override
    protected int foodEnergy() {
        return settings.herbivoreFoodEnergy();
    }


    @Override
    protected boolean isFood(Agent other) {
        return other instanceof Plant;
    }
```

```
    @Override
    protected int reproductionThreshold() {
        return settings.herbivoreReproductionThreshold();
    }

    @Override
    protected Agent createChild(int energy) {
        return new Herbivore(settings, energy);
    }
}
```

Файл src/main/java/life/model/Predator.java

```
package life.model;

import java.util.Random;

/** Хищник: охотится на травоядных. */
public class Predator extends Animal {

    public Predator(Settings settings) {
        this(settings, settings.predatorStartEnergy());
    }

    public Predator(Settings settings, int energy) {
        super(settings, energy);
    }

    /**
     * Ход хищника:
     * ЕСЛИ видит травоядное — идёт к нему (и съедает, если дошёл);
     * ИНАЧЕ бродит случайно.
     * Перед этим тратит энергию, после — размножается, если энергии хватает.
     */
    @Override
    public void act(World world, Random random) {
        if (!isAlive() || !spendEnergy(world)) {
            return;
        }
        Agent prey = nearest(look(world), this::isFood, random);
        if (prey != null) {
            moveTowards(world, prey.getPosition(), random);
        } else {
            wander(world, random);
        }
        tryReproduce(world, random);
    }

    @Override
    public char symbol() {
        return 'X';
    }

    @Override
    protected int stepCost() {
        return settings.predatorStepCost();
    }

    @Override
    protected int foodEnergy() {
        return settings.predatorFoodEnergy();
    }

    @Override
    protected boolean isFood(Agent other) {
        return other instanceof Herbivore;
```

```
    }

    @Override
    protected int reproductionThreshold() {
        return settings.predatorReproductionThreshold();
    }

    @Override
    protected Agent createChild(int energy) {
        return new Predator(settings, energy);
    }
}
```

Файл src/main/java/life/model/Statistics.java

```
package life.model;

/** Снимок численности на определённом шаге. */
public record Statistics(long step, int plants, int herbivores, int predators) {

    public int total() {
        return plants + herbivores + predators;
    }

    /** На поле никого не осталось — жизнь прекратилась. */
    public boolean allDead() {
        return total() == 0;
    }

    /** Хотя бы один вид вымер полностью. */
    public boolean anySpeciesExtinct() {
        return plants == 0 || herbivores == 0 || predators == 0;
    }
}
```

Файл src/main/java/life/model/Simulation.java

```
package life.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.Iterator;
import java.util.List;
import java.util.Random;

/**
 * Симуляция целиком: поле с жителями, генератор случайных чисел и счётчик шагов.
 * Снаружи доступны только «сделать шаг», статистика и поле для чтения.
 */
public class Simulation {

    private final Settings settings;
    private final World world;
    /** Один генератор на всю симуляцию: так ход игры полностью определяется seed из
настроек. */
    private final Random random;
    private long step;

    /** Новая симуляция: поле нужного размера со случайно расставленными жителями. */
    public Simulation(Settings settings) {
        this(settings, new World(settings.width(), settings.height()));
        populate();
    }

    /** Симуляция на готовом поле (для тестов): жители не расставляются. */
    Simulation(Settings settings, World world) {
```

```
        this.settings = settings;
        this.world = world;
        this.random = new Random(settings.seed());
    }


    /**
     * Один шаг времени: каждый житель, живой на начало шага, ходит один раз, в случайном
порядке.
     * Съеденный до своей очереди не ходит; родившийся в этом шаге ждёт следующего.
     */
    public void step() {
        List<Agent> order = new ArrayList<>(world.getAgents());
        Collections.shuffle(order, random);
        for (Agent agent : order) {
            if (agent.isAlive()) {
                agent.act(world, random);
            }
        }
        step++;
    }


    public Statistics statistics() {
        return new Statistics(step,
                world.count(Plant.class),
                world.count(Herbivore.class),
                world.count(Predator.class));
    }


    public ReadOnlyWorld getWorld() {
        return world;
    }


    public Settings getSettings() {
        return settings;
    }


    public long getStep() {
        return step;
    }


    /** Расставляет начальных жителей по случайным различным клеткам. */
    private void populate() {
        List<Position> cells = new ArrayList<>(settings.width() * settings.height());
        for (int y = 0; y < settings.height(); y++) {
            for (int x = 0; x < settings.width(); x++) {
                cells.add(new Position(x, y));
            }
        }
        Collections.shuffle(cells, random);
        Iterator<Position> free = cells.iterator();
        for (int i = 0; i < settings.initialPlants(); i++) {
            world.setAgent(free.next(), new Plant(settings));
        }
        for (int i = 0; i < settings.initialHerbivores(); i++) {
            world.setAgent(free.next(), new Herbivore(settings));
        }
        for (int i = 0; i < settings.initialPredators(); i++) {
            world.setAgent(free.next(), new Predator(settings));
        }
    }
}
```

Файл src/main/java/life/controller/SimulationController.java

```
package life.controller;


import life.model.ReadOnlyWorld;
import life.model.Settings;
import life.model.Simulation;
import life.model.Statistics;
```

```
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Deque;
import java.util.List;


/**
 * Управление симуляцией для оконной версии: старт, пауза, шаг, сброс, скорость.
 * Не знает ничего про JavaFX — окно только вызывает методы и рисует состояние,
 * поэтому всю логику кнопок можно проверить обычными тестами.
 * <p>
 * Время приходит снаружи через {@link #tick(long)} (окно вызывает его на каждом кадре),
 * а контроллер сам решает, сколько шагов сделать, исходя из скорости.
 */
public class SimulationController {


    public static final int MIN_SPEED = 1;
    public static final int MAX_SPEED = 500;
    public static final int DEFAULT_SPEED = 20;
    /** Больше шагов за один кадр не делаем, чтобы окно не подвисало. */
    public static final int MAX_STEPS_PER_TICK = 20;
    /** Сколько последних точек помнить для графика. */
    public static final int HISTORY_LIMIT = 1000;


    private static final long NANOS_PER_SECOND = 1_000_000_000L;


    private Settings settings;
    private Simulation simulation;
    private final Deque<Statistics> history = new ArrayDeque<>();


    private boolean running;
    private int speed = DEFAULT_SPEED;
    /** Время прошлого кадра; пока его нет (сразу после старта), первый кадр только
запоминает время. */
    private boolean hasLastTick;
    private long lastTick;
    /** Накопленное время × скорость: целая часть от деления на секунду — сколько шагов
пора сделать. */
    private long stepDebt;


    public SimulationController(Settings settings) {
        applySettings(settings);
    }


    /** Запустить. Повторное нажатие ничего не меняет; вымершее поле не запускается. */
    public void start() {
        if (running || simulation.statistics().allDead()) {
            return;
        }
        running = true;
        hasLastTick = false; // время, проведённое на паузе, не должно превратиться в шаги
        stepDebt = 0;
    }


    public void pause() {
        running = false;
    }


    /** Одна кнопка «Старт/Пауза». */
    public void toggleRunning() {
        if (running) {
            pause();
        } else {
            start();
        }
    }


    /** Кнопка «Шаг»: ставит на паузу и делает ровно один шаг. */
```

```
    public void stepOnce() {
        pause();
        if (!simulation.statistics().allDead()) {
            makeStep();
        }
    }


    /** Кнопка «Сброс»: то же поле с теми же настройками заново, на паузе. */
    public void reset() {
        pause();
        simulation = new Simulation(settings);
        history.clear();
        history.addLast(simulation.statistics());
    }


    /** Новые настройки из окна параметров: создаётся новое поле. */
    public void applySettings(Settings newSettings) {
        settings = newSettings;
        reset();
    }


    /** Скорость в шагах в секунду; значения вне допустимого диапазона прижимаются к
границе. */
    public void setSpeed(int stepsPerSecond) {
        speed = Math.clamp(stepsPerSecond, MIN_SPEED, MAX_SPEED);
    }


    /** Очередной кадр окна. nowNanos — текущее время в наносекундах. */
    public void tick(long nowNanos) {
        if (!running) {
            return;
        }
        if (!hasLastTick || nowNanos < lastTick) {
            hasLastTick = true;
            lastTick = nowNanos;
            return;
        }
        // Больше секунды за кадр не учитываем: иначе после долгого «замерзания» набежит
гигантский долг
        long elapsed = Math.min(nowNanos - lastTick, NANOS_PER_SECOND);
        lastTick = nowNanos;
        stepDebt += elapsed * speed;


        long due = stepDebt / NANOS_PER_SECOND;
        stepDebt %= NANOS_PER_SECOND;
        if (due > MAX_STEPS_PER_TICK) {
            due = MAX_STEPS_PER_TICK;
            stepDebt = 0; // не успели — не догоняем, просто идём медленнее
        }
        for (int i = 0; i < due; i++) {
            makeStep();
            if (simulation.statistics().allDead()) {
                pause();
                return;
            }
        }
    }


    public boolean isRunning() {
        return running;
    }


    public int getSpeed() {
        return speed;
    }


    public Settings getSettings() {
        return settings;
    }
```

```
    public ReadOnlyWorld getWorld() {
        return simulation.getWorld();
    }


    public Statistics getStatistics() {
        return simulation.statistics();
    }


    /** Последние точки численности для графика, от старых к новым. */
    public List<Statistics> getHistory() {
        return List.copyOf(history);
    }


    /** Что сообщить пользователю: жизнь прекратилась, какие виды вымерли или ничего. */
    public String getStatusMessage() {
        Statistics stats = simulation.statistics();
        if (stats.allDead()) {
            return "Жизнь прекратилась на шаге " + stats.step();
        }
        List<String> extinct = new ArrayList<>();
        if (stats.plants() == 0) {
            extinct.add("растения");
        }
        if (stats.herbivores() == 0) {
            extinct.add("травоядные");
        }
        if (stats.predators() == 0) {
            extinct.add("хищники");
        }
        return extinct.isEmpty() ? "" : "Вымерли: " + String.join(", ", extinct);
    }


    private void makeStep() {
        simulation.step();
        history.addLast(simulation.statistics());
        if (history.size() > HISTORY_LIMIT) {
            history.removeFirst();
        }
    }
}
```

Файл src/main/java/life/controller/SettingsForm.java

```
package life.controller;


import life.model.InvalidSettingsException;
import life.model.InvalidSettingsException.Problem;
import life.model.Settings;
import life.util.SafeText;


import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.regex.Pattern;


/**
 * Форма настроек без JavaFX: тексты из полей ввода превращаются в {@link Settings}
 * или в список понятных проблем. Окно параметров только показывает поля и результат.
 */
public final class SettingsForm {


    /** Поле формы: ключ (имя параметра в Settings) и подпись для человека. */
    public record Field(String key, String label) {
    }


    /** Результат разбора формы. */
```

```
    public sealed interface Result {
        record Valid(Settings settings) implements Result {
        }


        record Invalid(List<Problem> problems) implements Result {
        }
    }


    /** Поля в том же порядке, что и параметры в {@link Settings}. */
    public static final List<Field> FIELDS = List.of(
            new Field("width", "Ширина поля"),
            new Field("height", "Высота поля"),
            new Field("initialPlants", "Растений в начале"),
            new Field("initialHerbivores", "Травоядных в начале"),
            new Field("initialPredators", "Хищников в начале"),
            new Field("plantStartEnergy", "Растение: начальная энергия"),
            new Field("plantGrowth", "Растение: прирост энергии за шаг"),
            new Field("plantMaxEnergy", "Растение: максимум энергии"),
            new Field("plantReproductionThreshold", "Растение: порог размножения"),
            new Field("herbivoreStartEnergy", "Травоядное: начальная энергия"),
            new Field("herbivoreStepCost", "Травоядное: расход энергии за шаг"),
            new Field("herbivoreFoodEnergy", "Травоядное: энергия от растения"),
            new Field("herbivoreReproductionThreshold", "Травоядное: порог размножения"),
            new Field("predatorStartEnergy", "Хищник: начальная энергия"),
            new Field("predatorStepCost", "Хищник: расход энергии за шаг"),
            new Field("predatorFoodEnergy", "Хищник: энергия от травоядного"),
            new Field("predatorReproductionThreshold", "Хищник: порог размножения"),
            new Field("seed", "Зерно случайности (seed)"));


    /** Только ASCII-цифры, необязательный знак: «５» или «٣» числом не считаются. */
    private static final Pattern INTEGER = Pattern.compile("[+-]?[0-9]+");
    private static final int ECHO_LIMIT = 20;


    private SettingsForm() {
    }


    /** Значения настроек как тексты для полей формы, в порядке {@link #FIELDS}. */
    public static Map<String, String> toTexts(Settings s) {
        long[] values = {s.width(), s.height(),
                s.initialPlants(), s.initialHerbivores(), s.initialPredators(),
                s.plantStartEnergy(), s.plantGrowth(), s.plantMaxEnergy(),
s.plantReproductionThreshold(),
                s.herbivoreStartEnergy(), s.herbivoreStepCost(), s.herbivoreFoodEnergy(),
s.herbivoreReproductionThreshold(),
                s.predatorStartEnergy(), s.predatorStepCost(), s.predatorFoodEnergy(),
s.predatorReproductionThreshold(),
                s.seed()};
        Map<String, String> texts = new LinkedHashMap<>();
        for (int i = 0; i < FIELDS.size(); i++) {
            texts.put(FIELDS.get(i).key(), Long.toString(values[i]));
        }
        return texts;
    }


    /**
     * Сначала каждое поле читается как целое число; если хоть одно не прочиталось —
возвращаются эти ошибки.
     * Если все прочитались — проверяются правила самих настроек (размеры, пороги, влезут
ли жители).
     */
    public static Result parse(Map<String, String> texts) {
        List<Problem> problems = new ArrayList<>();
        long[] values = new long[FIELDS.size()];
        for (int i = 0; i < FIELDS.size(); i++) {
            Field field = FIELDS.get(i);
            String raw = texts.get(field.key());
            String text = raw == null ? "" : raw.strip();
            String error = null;
            if (text.isEmpty()) {
                error = "не заполнено";
            } else if (!INTEGER.matcher(text).matches()) {
```

```
                error = "нужно целое число, а указано \"" + SafeText.preview(text,
ECHO_LIMIT) + "\"";
            } else {
                try {
                    values[i] = field.key().equals("seed") ? Long.parseLong(text) :
Integer.parseInt(text);
                } catch (NumberFormatException e) {
                    error = "слишком большое число \"" + SafeText.preview(text, ECHO_LIMIT)
+ "\"";
                }
            }
            if (error != null) {
                problems.add(new Problem(field.key(), field.label() + ": " + error));
            }
        }
        if (!problems.isEmpty()) {
            return new Result.Invalid(List.copyOf(problems));
        }
        try {
            return new Result.Valid(Settings.builder()
                    .width((int) values[0]).height((int) values[1])
                    .initialPlants((int) values[2]).initialHerbivores((int)
values[3]).initialPredators((int) values[4])
                    .plantStartEnergy((int) values[5]).plantGrowth((int) values[6])
                    .plantMaxEnergy((int) values[7]).plantReproductionThreshold((int)
values[8])
                    .herbivoreStartEnergy((int) values[9]).herbivoreStepCost((int)
values[10])
                    .herbivoreFoodEnergy((int)
values[11]).herbivoreReproductionThreshold((int) values[12])
                    .predatorStartEnergy((int) values[13]).predatorStepCost((int)
values[14])
                    .predatorFoodEnergy((int)
values[15]).predatorReproductionThreshold((int) values[16])
                    .seed(values[17])
                    .build());
        } catch (InvalidSettingsException e) {
            return new Result.Invalid(e.problems());
        }
    }
}
```

Файл src/main/java/life/controller/SpeedScale.java

```
package life.controller;


/**
 * Перевод положения слайдера скорости в шаги в секунду и обратно.
 * Шкала логарифмическая: на линейной от 1 до 500 медленные скорости заняли бы крошечный
кусочек слайдера.
 */
public final class SpeedScale {


    public static final double SLIDER_MIN = 0;
    public static final double SLIDER_MAX = 100;


    private static final double LOG_MIN = Math.log(SimulationController.MIN_SPEED);
    private static final double LOG_MAX = Math.log(SimulationController.MAX_SPEED);


    private SpeedScale() {
    }


    /** Положение слайдера → скорость. */
    public static int toSpeed(double slider) {
        double t = (slider - SLIDER_MIN) / (SLIDER_MAX - SLIDER_MIN);
        long speed = Math.round(Math.exp(LOG_MIN + t * (LOG_MAX - LOG_MIN)));
        // За краями слайдера exp даёт почти 0 или бесконечность, NaN округляется в 0 — всё
прижимается к диапазону
        return (int) Math.clamp(speed, SimulationController.MIN_SPEED,
SimulationController.MAX_SPEED);
    }
```

```
    /** Скорость → положение слайдера. */
    public static double toSlider(int speed) {
        int clamped = Math.clamp(speed, SimulationController.MIN_SPEED,
SimulationController.MAX_SPEED);
        return SLIDER_MIN + (Math.log(clamped) - LOG_MIN) / (LOG_MAX - LOG_MIN) *
(SLIDER_MAX - SLIDER_MIN);
    }
}
```

Файл src/main/java/life/util/SafeText.java

```
package life.util;


/** Безопасный показ введённого пользователем текста внутри сообщений. */
public final class SafeText {


    private SafeText() {
    }


    /**
     * Текст для сообщения об ошибке: не длиннее limit символов (дальше «...»),
     * управляющие символы заменены на «?», чтобы они не испортили экран.
     */
    public static String preview(String text, int limit) {
        StringBuilder sb = new StringBuilder();
        text.codePoints()
                .limit(limit)
                .forEach(cp -> sb.appendCodePoint(Character.isISOControl(cp) ? '?' : cp));
        if (text.codePointCount(0, text.length()) > limit) {
            sb.append("...");
        }
        return sb.toString();
    }
}
```

Файл src/main/java/life/view/console/ConsoleMain.java

```
package life.view.console;


import life.model.Settings;
import life.model.Simulation;


import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.nio.charset.Charset;


/** Точка входа консольной версии. */
public final class ConsoleMain {


    private ConsoleMain() {
    }


    public static void main(String[] args) {
        // Кодировка клавиатурного ввода консоли (на Windows это обычно не UTF-8).
        // System.out уже настроен Java на кодировку консоли сам.
        Charset inputCharset = Charset.forName(
                System.getProperty("stdin.encoding", Charset.defaultCharset().name()),
Charset.defaultCharset());
        BufferedReader in = new BufferedReader(new InputStreamReader(System.in,
inputCharset));
        new ConsoleApp(new Simulation(Settings.defaults()), in, System.out).run();
    }
}
```

Файл src/main/java/life/view/console/ConsoleApp.java

```
package life.view.console;


import life.model.Simulation;


import java.io.BufferedReader;
import java.io.IOException;
import java.io.PrintStream;
import java.util.function.LongSupplier;


/** Консольная версия: показывает поле, читает команды и выполняет их, пока пользователь не
выйдет. */
public class ConsoleApp {


    static final String HELP = """
            Команды:
              Enter     - один шаг
              число N   - N шагов подряд (от 1 до %d)
              h или ?   - эта справка
              q         - выход
            """.formatted(CommandParser.MAX_STEPS);


    /** Как часто печатать прогресс долгой команды: раз в секунду. */
    private static final long PROGRESS_INTERVAL_NANOS = 1_000_000_000L;


    private final Simulation simulation;
    private final BufferedReader in;
    private final PrintStream out;
    /** Текущее время в наносекундах. Снаружи, чтобы тесты могли подставить поддельные
часы. */
    private final LongSupplier clock;


    public ConsoleApp(Simulation simulation, BufferedReader in, PrintStream out) {
        this(simulation, in, out, System::nanoTime);
    }


    ConsoleApp(Simulation simulation, BufferedReader in, PrintStream out, LongSupplier
clock) {
        this.simulation = simulation;
        this.in = in;
        this.out = out;
        this.clock = clock;
    }


    public void run() {
        out.println("Искусственная жизнь - консольная версия");
        out.print(HELP);
        show();
        while (true) {
            out.print("> ");
            out.flush();
            String line = readLine();
            switch (CommandParser.parse(line)) {
                case Command.Quit quit -> {
                    if (line == null) {
                        out.println(); // после «> » без перевода строки
                    }
                    out.println("До свидания!");
                    return;
                }
                case Command.Help help -> out.print(HELP);
                case Command.Invalid invalid -> out.println("Ошибка: " +
invalid.message());
                case Command.Step step -> makeSteps(step.count());
            }
        }
    }
```

```
    /** Следующая строка ввода или null, если ввод закончился или сломался. */
    private String readLine() {
        try {
            return in.readLine();
        } catch (IOException e) {
            out.println();
            out.println("Не удалось прочитать ввод: " + e.getMessage());
            return null;
        }
    }


    /** Делает до count шагов, но останавливается, как только на поле никого не осталось.
*/
    private void makeSteps(int count) {
        if (simulation.statistics().allDead()) {
            out.println("Жизнь прекратилась - шагать больше некуда. Выход: q");
            return;
        }
        // Долгая команда не должна выглядеть зависшей: раз в секунду сообщаем, сколько
сделано.
        long lastReport = clock.getAsLong();
        for (int i = 1; i <= count && !simulation.statistics().allDead(); i++) {
            simulation.step();
            long now = clock.getAsLong();
            if (i < count && now - lastReport >= PROGRESS_INTERVAL_NANOS) {
                out.println("...выполнено " + i + " из " + count);
                lastReport = now;
            }
        }
        show();
        if (simulation.statistics().allDead()) {
            out.println("Жизнь прекратилась на шаге " + simulation.getStep() + ".");
        }
    }


    private void show() {
        out.print(ConsoleRenderer.render(simulation.getWorld(), simulation.statistics()));
    }
}
```

Файл src/main/java/life/view/console/Command.java

```
package life.view.console;


/** Команда, которую пользователь ввёл в консоли. */
public sealed interface Command {


    /** Сделать count шагов подряд. */
    record Step(int count) implements Command {
    }


    /** Показать справку. */
    record Help() implements Command {
    }


    /** Выйти из программы. */
    record Quit() implements Command {
    }


    /** Ввод не распознан; message — что сказать пользователю. */
    record Invalid(String message) implements Command {
    }
}
```

Файл src/main/java/life/view/console/CommandParser.java

```
package life.view.console;
```

```
import life.util.SafeText;


import java.util.Locale;
import java.util.Set;
import java.util.regex.Pattern;


/**
 * Разбирает строку, введённую пользователем, в команду. Никогда не бросает исключений:
 * на любой мусор отвечает {@link Command.Invalid} с понятным объяснением.
 * В сообщениях только ASCII-знаки препинания — их покажет любая консоль, в том числе
Windows.
 */
public final class CommandParser {


    public static final int MAX_STEPS = 100_000;


    /** Сколько символов введённого текста повторять в сообщении об ошибке. */
    private static final int ECHO_LIMIT = 20;


    private static final Set<String> QUIT_WORDS = Set.of("q", "quit", "exit", "выход",
"й");
    private static final Set<String> HELP_WORDS = Set.of("h", "help", "?", "помощь", "р");


    /** Только ASCII-цифры: «5» или «٥» — не число для нас. */
    private static final Pattern DIGITS = Pattern.compile("[0-9]+");
    /** Похоже на число, но не целое положительное: со знаком или с дробной частью. */
    private static final Pattern NUMBER_LIKE = Pattern.compile("(?=.*[0-9])[+-]?[0-
9]*([.,][0-9]*)?");


    private CommandParser() {
    }


    public static Command parse(String line) {
        if (line == null) {
            return new Command.Quit(); // конец ввода (Ctrl+D или закончился файл)
        }
        // BOM появляется в начале файла, сохранённого в Блокноте Windows
        String text = line.replace("\uFEFF", "").strip();
        String word = text.toLowerCase(Locale.ROOT);


        if (word.isEmpty()) {
            return new Command.Step(1);
        }
        if (QUIT_WORDS.contains(word)) {
            return new Command.Quit();
        }
        if (HELP_WORDS.contains(word)) {
            return new Command.Help();
        }
        if (DIGITS.matcher(word).matches()) {
            return steps(word);
        }
        if (NUMBER_LIKE.matcher(word).matches()) {
            return new Command.Invalid("число шагов должно быть целое, от 1 до " +
MAX_STEPS
                    + ", а введено \"" + echo(text) + "\"");
        }
        return new Command.Invalid("неизвестная команда \"" + echo(text)
                + "\". Enter - шаг, число - несколько шагов, h - справка, q - выход");
    }


    private static Command steps(String digits) {
        String significant = digits.replaceFirst("^0+(?=.)", "");
        // Больше шести цифр — заведомо больше максимума; так огромное число не переполнит
int
        if (significant.length() > 6) {
            return outOfRange(digits);
```

```
        }
        int count = Integer.parseInt(significant);
        if (count == 0 || count > MAX_STEPS) {
            return outOfRange(digits);
        }
        return new Command.Step(count);
    }

    private static Command outOfRange(String digits) {
        return new Command.Invalid("число шагов должно быть от 1 до " + MAX_STEPS + ", а
введено \"" + echo(digits) + "\"");
    }

    private static String echo(String text) {
        return SafeText.preview(text, ECHO_LIMIT);
    }
}
```

Файл src/main/java/life/view/console/ConsoleRenderer.java

```
package life.view.console;

import life.model.Agent;
import life.model.ReadOnlyWorld;
import life.model.Statistics;

import java.util.ArrayList;
import java.util.List;

/** Рисует поле и счётчики текстом для консоли. Только ASCII-символы рамки — их покажет
любая консоль. */
public final class ConsoleRenderer {

    /** Больше этого поле в консоль не выводится целиком — только левый верхний угол. */
    public static final int MAX_COLUMNS = 120;
    public static final int MAX_ROWS = 60;

    private ConsoleRenderer() {
    }

    public static String render(ReadOnlyWorld world, Statistics stats) {
        int columns = Math.min(world.getWidth(), MAX_COLUMNS);
        int rows = Math.min(world.getHeight(), MAX_ROWS);
        StringBuilder sb = new StringBuilder();

        sb.append("Шаг ").append(stats.step())
                .append(" | растения P: ").append(stats.plants())
                .append(" | травоядные T: ").append(stats.herbivores())
                .append(" | хищники X: ").append(stats.predators())
                .append('\n');

        String border = "+" + "-".repeat(columns) + "+\n";
        sb.append(border);
        for (int y = 0; y < rows; y++) {
            sb.append('|');
            for (int x = 0; x < columns; x++) {
                Agent agent = world.getAgent(x, y);
                sb.append(agent == null ? '.' : agent.symbol());
            }
            sb.append("|\n");
        }
        sb.append(border);

        if (columns < world.getWidth() || rows < world.getHeight()) {
            sb.append("(показан фрагмент ").append(columns).append('x').append(rows)
                    .append(" из
").append(world.getWidth()).append('x').append(world.getHeight()).append(")\n");
```

```
        }

        List<String> extinct = new ArrayList<>();
        if (stats.plants() == 0) {
            extinct.add("растения");
        }
        if (stats.herbivores() == 0) {
            extinct.add("травоядные");
        }
        if (stats.predators() == 0) {
            extinct.add("хищники");
        }
        if (!extinct.isEmpty()) {
            sb.append("Вымерли: ").append(String.join(", ", extinct)).append('\n');
        }
        return sb.toString();
    }
}
```

Файл src/main/java/life/view/gui/LifeApp.java

```
package life.view.gui;


import javafx.animation.AnimationTimer;
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.Separator;
import javafx.scene.control.Slider;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.stage.Stage;
import life.controller.SimulationController;
import life.controller.SpeedScale;
import life.model.Settings;
import life.model.Statistics;


/**
 * Главное окно: поле в центре, управление справа, график снизу.
 * Игровой цикл — таймер JavaFX (AnimationTimer): на каждом кадре контроллер решает,
 * сколько шагов сделать, а окно перерисовывает то, что изменилось.
 */
public class LifeApp extends Application {


    /** График перерисовываем не чаще 4 раз в секунду — он дороже поля. */
    private static final long CHART_INTERVAL_NANOS = 250_000_000L;


    private SimulationController controller;
    private AnimationTimer timer;


    private final WorldCanvas canvas = new WorldCanvas();
    private final PopulationChart chart = new PopulationChart();
    private final Button startPause = new Button();
    private final Button step = new Button("Шаг");
    private final Button reset = new Button("Сброс");
    private final Button parameters = new Button("Параметры…");
    private final Slider speed = new Slider(SpeedScale.SLIDER_MIN, SpeedScale.SLIDER_MAX,
0);
    private final Label speedLabel = new Label();
    private final Label stepLabel = new Label();
    private final Label plantsLabel = new Label();
    private final Label herbivoresLabel = new Label();
    private final Label predatorsLabel = new Label();
    private final Label statusLabel = new Label();
```

```
private long lastDrawnStep = -1;
private long lastChartUpdate;


static String stylesheet() {
    return LifeApp.class.getResource("app.css").toExternalForm();
}


@Override
public void start(Stage stage) {
    controller = new SimulationController(Settings.defaults());


    BorderPane root = new BorderPane();
    root.setCenter(canvas);
    root.setRight(buildControls());
    chart.setPrefHeight(230);
    root.setBottom(chart);
    BorderPane.setMargin(canvas, new Insets(8));


    Scene scene = new Scene(root, 1200, 820);
    scene.getStylesheets().add(stylesheet());
    stage.setTitle("Искусственная жизнь");
    stage.setMinWidth(820);
    stage.setMinHeight(620);
    stage.setScene(scene);


    timer = new AnimationTimer() {
        @Override
        public void handle(long now) {
            controller.tick(now);
            refresh(false, now);
        }
    };
    refresh(true, System.nanoTime());
    stage.show();
    timer.start();
}


@Override
public void stop() {
    if (timer != null) {
        timer.stop();
    }
}


private VBox buildControls() {
    Label title = new Label("Управление");
    title.getStyleClass().add("title");


    startPause.setId("startPause");
    step.setId("step");
    reset.setId("reset");
    parameters.setId("parameters");
    speed.setId("speed");
    for (Button button : new Button[]{startPause, step, reset, parameters}) {
        button.setMaxWidth(Double.MAX_VALUE);
    }


    startPause.setOnAction(e -> command(controller::toggleRunning));
    step.setOnAction(e -> command(controller::stepOnce));
    reset.setOnAction(e -> command(controller::reset));
    parameters.setOnAction(e -> openParameters());


    speed.setValue(SpeedScale.toSlider(controller.getSpeed()));
    speed.valueProperty().addListener((observable, oldValue, newValue) -> {
        controller.setSpeed(SpeedScale.toSpeed(newValue.doubleValue()));
        updateSpeedLabel();
    });
```

```
        updateSpeedLabel();

        statusLabel.getStyleClass().add("status");
        statusLabel.setWrapText(true);
        statusLabel.setMaxWidth(220);

        VBox counters = new VBox(6,
                stepLabel,
                counter(WorldCanvas.PLANT, plantsLabel),
                counter(WorldCanvas.HERBIVORE, herbivoresLabel),
                counter(WorldCanvas.PREDATOR, predatorsLabel));

        VBox box = new VBox(10,
                title, startPause, step, reset,
                new Separator(), speedLabel, speed,
                new Separator(), counters, statusLabel,
                new Separator(), parameters);
        box.getStyleClass().add("panel");
        box.setPrefWidth(240);
        return box;
    }

    private static HBox counter(Color color, Label label) {
        Rectangle square = new Rectangle(14, 14, color);
        square.setStroke(Color.web("#455a64"));
        HBox row = new HBox(8, square, label);
        row.setAlignment(Pos.CENTER_LEFT);
        label.getStyleClass().add("counter");
        return row;
    }

    /** Нажатие кнопки: выполнить действие и сразу показать результат. */
    private void command(Runnable action) {
        action.run();
        refresh(true, System.nanoTime());
    }

    private void openParameters() {
        controller.pause();
        new
SettingsDialog(controller.getSettings()).showAndWait().ifPresent(controller::applySettings)
;
        refresh(true, System.nanoTime());
    }

    private void updateSpeedLabel() {
        speedLabel.setText("Скорость: " + controller.getSpeed() + " шаг./с");
    }

    private void refresh(boolean force, long now) {
        Statistics stats = controller.getStatistics();
        boolean changed = stats.step() != lastDrawnStep;
        if (force || changed) {
            canvas.draw(controller.getWorld());
            stepLabel.setText("Шаг: " + stats.step());
            plantsLabel.setText("Растения: " + stats.plants());
            herbivoresLabel.setText("Травоядные: " + stats.herbivores());
            predatorsLabel.setText("Хищники: " + stats.predators());
            statusLabel.setText(controller.getStatusMessage());
            lastDrawnStep = stats.step();
        }
        if (force || (changed && now - lastChartUpdate >= CHART_INTERVAL_NANOS)) {
            chart.update(controller.getHistory());
            lastChartUpdate = now;
        }
        boolean dead = stats.allDead();
        startPause.setText(controller.isRunning() ? "Пауза" : "Старт");
        startPause.setDisable(dead);
        step.setDisable(dead);
    }
```

```
}
```

Файл src/main/java/life/view/gui/WorldCanvas.java

```
package life.view.gui;


import javafx.scene.canvas.Canvas;
import javafx.scene.canvas.GraphicsContext;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import life.model.Agent;
import life.model.Herbivore;
import life.model.Plant;
import life.model.Predator;
import life.model.ReadOnlyWorld;


/** Поле цветными клетками. Растягивается под размер окна, клетки остаются квадратными. */
public class WorldCanvas extends Pane {


    static final Color BACKGROUND = Color.web("#eceff1");
    static final Color EMPTY = Color.web("#263238");
    static final Color PLANT = Color.web("#43a047");
    static final Color HERBIVORE = Color.web("#fdd835");
    static final Color PREDATOR = Color.web("#e53935");


    private final Canvas canvas = new Canvas();
    private ReadOnlyWorld world;


    public WorldCanvas() {
        getChildren().add(canvas);
    }


    /** Нарисовать поле (запоминается — при изменении размера окна перерисуется само). */
    public void draw(ReadOnlyWorld world) {
        this.world = world;
        redraw();
    }


    static Color colorOf(Agent agent) {
        return switch (agent) {
            case Plant plant -> PLANT;
            case Herbivore herbivore -> HERBIVORE;
            case Predator predator -> PREDATOR;
            default -> Color.GRAY;
        };
    }


    @Override
    protected void layoutChildren() {
        canvas.setWidth(getWidth());
        canvas.setHeight(getHeight());
        redraw();
    }


    // Размер холста не должен влиять на желаемый размер панели — иначе окно нельзя
уменьшить.
    @Override
    protected double computePrefWidth(double height) {
        return 800;
    }


    @Override
    protected double computePrefHeight(double width) {
        return 480;
    }


    @Override
```

```
    protected double computeMinWidth(double height) {
        return 100;
    }

    @Override
    protected double computeMinHeight(double width) {
        return 60;
    }

    private void redraw() {
        double width = canvas.getWidth();
        double height = canvas.getHeight();
        GraphicsContext g = canvas.getGraphicsContext2D();
        g.setFill(BACKGROUND);
        g.fillRect(0, 0, width, height);
        if (world == null || width <= 0 || height <= 0) {
            return;
        }

        double cell = Math.min(width / world.getWidth(), height / world.getHeight());
        double left = (width - cell * world.getWidth()) / 2;
        double top = (height - cell * world.getHeight()) / 2;
        // Тонкий зазор между клетками, если они достаточно крупные
        double size = cell >= 5 ? cell - 1 : cell;

        g.setFill(EMPTY);
        g.fillRect(left, top, cell * world.getWidth(), cell * world.getHeight());
        for (int y = 0; y < world.getHeight(); y++) {
            for (int x = 0; x < world.getWidth(); x++) {
                Agent agent = world.getAgent(x, y);
                if (agent != null) {
                    g.setFill(colorOf(agent));
                    g.fillRect(left + x * cell, top + y * cell, size, size);
                }
            }
        }
    }
}
```

Файл src/main/java/life/view/gui/PopulationChart.java

```
package life.view.gui;

import javafx.scene.chart.LineChart;
import javafx.scene.chart.NumberAxis;
import javafx.scene.chart.XYChart;
import javafx.util.StringConverter;
import life.model.Statistics;

import java.util.ArrayList;
import java.util.List;

/** График численности трёх видов по шагам. */
public class PopulationChart extends LineChart<Number, Number> {

    /** Больше точек на линию не рисуем: график остаётся быстрым при любой длине истории.
*/
    private static final int MAX_POINTS = 300;

    private final XYChart.Series<Number, Number> plants = new XYChart.Series<>();
    private final XYChart.Series<Number, Number> herbivores = new XYChart.Series<>();
    private final XYChart.Series<Number, Number> predators = new XYChart.Series<>();

    public PopulationChart() {
        super(new NumberAxis(), new NumberAxis());
        setAnimated(false);
        setCreateSymbols(false);
        getXAxis().setLabel("Шаг");
```

```
        getYAxis().setLabel("Численность");
        NumberAxis steps = (NumberAxis) getXAxis();
        steps.setForceZeroInRange(false);
        // Номер шага — целое число: «0», а не «0.00»
        steps.setTickLabelFormatter(new StringConverter<>() {
            @Override
            public String toString(Number value) {
                return Long.toString(Math.round(value.doubleValue()));
            }

            @Override
            public Number fromString(String text) {
                return Long.parseLong(text);
            }
        });

        plants.setName("Растения");
        herbivores.setName("Травоядные");
        predators.setName("Хищники");
        getData().addAll(List.of(plants, herbivores, predators));
    }

    public void update(List<Statistics> history) {
        List<XYChart.Data<Number, Number>> p = new ArrayList<>();
        List<XYChart.Data<Number, Number>> h = new ArrayList<>();
        List<XYChart.Data<Number, Number>> x = new ArrayList<>();
        int stride = Math.max(1, (int) Math.ceil(history.size() / (double) MAX_POINTS));
        for (int i = 0; i < history.size(); i++) {
            // берём каждую stride-ю точку, но последнюю — всегда
            if (i % stride != 0 && i != history.size() - 1) {
                continue;
            }
            Statistics s = history.get(i);
            p.add(new XYChart.Data<>(s.step(), s.plants()));
            h.add(new XYChart.Data<>(s.step(), s.herbivores()));
            x.add(new XYChart.Data<>(s.step(), s.predators()));
        }
        plants.getData().setAll(p);
        herbivores.getData().setAll(h);
        predators.getData().setAll(x);
    }
}
```

Файл src/main/java/life/view/gui/SettingsDialog.java

```
package life.view.gui;

import javafx.event.ActionEvent;
import javafx.geometry.Insets;
import javafx.scene.control.Button;
import javafx.scene.control.ButtonBar;
import javafx.scene.control.ButtonType;
import javafx.scene.control.Dialog;
import javafx.scene.control.Label;
import javafx.scene.control.ScrollPane;
import javafx.scene.control.TextField;
import javafx.scene.layout.ColumnConstraints;
import javafx.scene.layout.GridPane;
import javafx.scene.layout.HBox;
import javafx.scene.layout.Region;
import javafx.scene.layout.VBox;
import life.controller.SettingsForm;
import life.model.InvalidSettingsException.Problem;
import life.model.Settings;

import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Random;
import java.util.Set;
```

```
/**
 * Окно параметров. Проверка введённого — в {@link SettingsForm};
 * здесь только поля, подсветка ошибок и кнопки. С ошибками окно не закрывается.
 */
public class SettingsDialog extends Dialog<Settings> {


    private static final ButtonType APPLY = new ButtonType("Применить",
ButtonBar.ButtonData.OK_DONE);
    private static final ButtonType CANCEL = new ButtonType("Отмена",
ButtonBar.ButtonData.CANCEL_CLOSE);
    /** Проблема «жители не влезли» относится сразу к трём полям. */
    private static final Set<String> COUNT_FIELDS = Set.of("initialPlants",
"initialHerbivores", "initialPredators");


    private final Map<String, TextField> inputs = new LinkedHashMap<>();
    private final Label errors = new Label();
    private Settings accepted;


    public SettingsDialog(Settings current) {
        setTitle("Параметры симуляции");
        setHeaderText("После применения поле создаётся заново.");


        GridPane grid = new GridPane();
        grid.setHgap(10);
        grid.setVgap(6);
        grid.setPadding(new Insets(10));
        // Подписи никогда не обрезаются многоточием: столбец не уже самой длинной подписи
        ColumnConstraints labelColumn = new ColumnConstraints();
        labelColumn.setMinWidth(Region.USE_PREF_SIZE);
        grid.getColumnConstraints().addAll(labelColumn, new ColumnConstraints(), new
ColumnConstraints());
        Map<String, String> texts = SettingsForm.toTexts(current);
        int row = 0;
        for (SettingsForm.Field field : SettingsForm.FIELDS) {
            TextField input = new TextField(texts.get(field.key()));
            input.setId("field-" + field.key());
            input.setPrefColumnCount(12);
            inputs.put(field.key(), input);
            grid.add(new Label(field.label()), 0, row);
            grid.add(input, 1, row);
            row++;
        }
        Button randomSeed = new Button("Случайный seed");
        randomSeed.setOnAction(e -> inputs.get("seed").setText(Long.toString(new
Random().nextLong())));
        randomSeed.setMinWidth(Region.USE_PREF_SIZE);
        grid.add(randomSeed, 2, row - 1);


        Button defaults = new Button("По умолчанию");
        defaults.setOnAction(e -> SettingsForm.toTexts(Settings.defaults())
                .forEach((key, text) -> inputs.get(key).setText(text)));


        errors.getStyleClass().add("error-text");
        errors.setWrapText(true);
        errors.setMaxWidth(560);
        // Блок ошибок растёт по высоте под все строки, а не обрезается
        errors.setMinHeight(Region.USE_PREF_SIZE);


        ScrollPane scroll = new ScrollPane(grid);
        scroll.setFitToWidth(true);
        scroll.setPrefViewportHeight(610);
        VBox content = new VBox(10, scroll, new HBox(defaults), errors);
        getDialogPane().setContent(content);
        getDialogPane().getButtonTypes().addAll(APPLY, CANCEL);
        getDialogPane().getStylesheets().add(LifeApp.stylesheet());


        Button apply = (Button) getDialogPane().lookupButton(APPLY);
        apply.addEventFilter(ActionEvent.ACTION, event -> {
```

```
            if (!tryAccept()) {
                event.consume(); // есть ошибки — окно остаётся открытым
            }
        });
        setResultConverter(button -> button == APPLY ? accepted : null);
    }

    private boolean tryAccept() {
        Map<String, String> texts = new HashMap<>();
        inputs.forEach((key, input) -> texts.put(key, input.getText()));
        inputs.values().forEach(input -> input.getStyleClass().remove("invalid"));

        switch (SettingsForm.parse(texts)) {
            case SettingsForm.Result.Valid valid -> {
                accepted = valid.settings();
                errors.setText("");
                return true;
            }
            case SettingsForm.Result.Invalid invalid -> {
                showProblems(invalid.problems());
                return false;
            }
        }
    }

    private void showProblems(List<Problem> problems) {
        StringBuilder text = new StringBuilder();
        for (Problem problem : problems) {
            text.append("• ").append(problem.message()).append('\n');
            Set<String> fields = problem.field().equals("initialCount") ? COUNT_FIELDS :
Set.of(problem.field());
            for (String key : fields) {
                TextField input = inputs.get(key);
                if (input != null && !input.getStyleClass().contains("invalid")) {
                    input.getStyleClass().add("invalid");
                }
            }
        }
        errors.setText(text.toString().strip());
    }
}
```

Файл src/main/resources/life/view/gui/app.css

```
.root {
    -fx-font-size: 13px;
}

.panel {
    -fx-padding: 14;
    -fx-background-color: #f5f6f8;
    -fx-border-color: #dde1e6;
    -fx-border-width: 0 0 0 1;
}

.title {
    -fx-font-size: 16px;
    -fx-font-weight: bold;
}

.counter {
    -fx-font-size: 14px;
}

.status {
    -fx-text-fill: #c62828;
    -fx-font-weight: bold;
}
```

```
.error-text {
    -fx-text-fill: #c62828;
}


.text-field.invalid {
    -fx-border-color: #e53935;
    -fx-border-width: 2;
    -fx-border-radius: 3;
}


/* Цвета линий графика совпадают с цветами клеток на поле */
.chart-series-line {
    -fx-stroke-width: 2px;
}


.default-color0.chart-series-line { -fx-stroke: #43a047; }
.default-color1.chart-series-line { -fx-stroke: #f9a825; }
.default-color2.chart-series-line { -fx-stroke: #e53935; }


.default-color0.chart-legend-item-symbol { -fx-background-color: #43a047; }
.default-color1.chart-legend-item-symbol { -fx-background-color: #f9a825; }
.default-color2.chart-legend-item-symbol { -fx-background-color: #e53935; }
```